Codes Used on the Course

03LogicalOperators_App.java
04TernaryOperator_App.java
05TypeComparisonOperator_App.java
05TypeComparisonOperator_ChildClass.java
07AutoboxingUnboxing_App.java
08WrapperClassesUsefulMethods_App.java
09NumberAndBooleanClassesMethods_App.java
10CharacterClassMethods_App.java
12GarbageCollection_App.java
12GarbageCollection_Car.java
14Heap_App.java
16ConstructorOverloading_App.java
16ConstructorOverloading_House.java
17MethodOverloading_App.java
20StringComparison_App.java
21StringIntern_App.java
23StringMethods_App.java
25StringSBuilderSBuffer_App.java
26StringBuilderMethods_App.java
28Enum_App.java
28Enum_Day.java
30DatesAndTimesClassic_App.java
31DatesAndTimesNew_App.java
32DatesAndTimesNew2_App.java
34NestedIf_App.java
36NestedLoops_App.java
38ArraysMethods_App.java
39PassingReturningArrays_App.java
41MultidimensionalArrays_App.java
43Varargs_App.java
45ArrayListSorting_App.java
46ArrayListMethods_App.java
48GenericsMethods_App.java
49GenericsClass_App.java
50GenericsWildcardsUpperBounded_App.java
51GenericsWildcardsLowerBounded_App.java
52GenericsWildcardsUnbounded_App.java
54Lists_App.java
55Queues_PriorityQueue_AccessCard.java
55Queues_PriorityQueue_App.java
56Queues_ArrayDeque_App.java
57Sets_App.java
58Maps_App.java
60LocalInnerClass_App.java
60LocalInnerClass_OuterClass.java
61MemberInnerClass_App.java
61MemberInnerClass_OuterClass.java
62StaticNestedClass_App.java
62StaticNestedClass_OuterClass.java
63AnonymousInnerClass_AbstractClassForAnonymousClass.java
63AnonymousInnerClass_App.java
63AnonymousInnerClass_InterfaceForAnonymousClass.java

63AnonymousInnerClass_WithoutAnonimous.java
64Exceptions_App.java
65ExceptionsA_App.java
65ExceptionsB_App.java
65ExceptionsC_App.java
65ExceptionsC_ExceptionChild.java
65ExceptionsC_ExceptionParent.java
68InheritanceTypes_App.java
68InheritanceTypes_Child.java
68InheritanceTypes_Child2.java
68InheritanceTypes_Grandchild.java
68InheritanceTypes_HybridInheritanceClass.java
68InheritanceTypes_Interface1.java
68InheritanceTypes_Interface2.java
68InheritanceTypes_Interface3.java
68InheritanceTypes_MultipleInheritanceClass.java
68InheritanceTypes_Parent.java
69Polymorphism_App.java
69Polymorphism_Child.java
69Polymorphism_GrandChild.java
69Polymorphism_Parent.java
70AdvancedEncapsulation_App.java
70AdvancedEncapsulation_Person.java
71EncapsulationWithMutableClasses_App.java
71EncapsulationWithMutableClasses_Cars.java
72JavaBeans_App.java
72JavaBeans_JavaBeanExample.java
73AbstractionMoreAboutInterface_AnotherInterface.java
73AbstractionMoreAboutInterface_App.java
73AbstractionMoreAboutInterface_ModernInterface.java
74AbstractionNestedInterface_App.java
74AbstractionNestedInterface_OuterClass.java
74AbstractionNestedInterface_OuterInterface.java
76Serialization_Car.java
76Serialization_Ford.java
76Serialization_Serialization.java
78LambdaExpression_App.java
78LambdaExpression_Greatings.java
78LambdaExpression_Sum.java
79FunctionalInterface_App.java
79FunctionalInterface_Sum.java
81MethodReference_App.java
81MethodReference_MessageClass.java
82MethodReferenceVsLambda_App.java
82MethodReferenceVsLambda_Sum.java
83forEachWithLambdaAndMethodReference_App.java
85OptionalOverviewExamples_OptionalClass.java
85OptionalOverviewExamples_Person.java
86A_NonOptionalExample_NonOptionalExample.java
86A_NonOptionalExample_Passport.java
86A_NonOptionalExample_Person.java
86A_NonOptionalExample_Stamp.java
86B_OptionalExample_OptionalExample.java
86B_OptionalExample_Passport.java

86B_OptionalExample_Person.java
86B_OptionalExample_Stamp.java
88OperationOfStream_App.java
89StreamMethods_App.java
90FilteringMapWithStream_App.java
92SequentialVsParallelStream_App.java
94MultithreadingCreating1ThreadClass_App.java
94MultithreadingCreating2RunnableInterface_App.java
95MultithreadingExample1_App.java
95MultithreadingExample2_App.java
97Synchronized_App.java
98SynchronizedBlock_App.java
99SynchronizedWaitNotify4_App.java
99SynchronizedWaitNotify_App.java

```
##--> 03LogicalOperators_App.java<-- ##

package com.udemy.app;

public class App {

      /**         Logical Operators
       *
       *          &&    Logical AND      'A' and 'B' are true
       *          ||    Logical OR       'A' or 'B' are true
       *          !     Logical NOT      Reverse the logical state
       *          ^     Logical XOR      'A' or 'B' are true BUT not both
       *
       **/

      public static void main(String[] args) {

            int a = 10;
            int b = 20;

            //    TRUE && TRUE > TRUE
            if(a < 15 && b > 15) {
                  System.out.println("True");
            }
            else {
                  System.out.println("False");
            }

            //    TRUE || FALSE > TRUE
            if(a < 15 || b > 1000) {
                  System.out.println("True");
            }
            else {
                  System.out.println("False");
            }

            //    !TRUE > FALSE
            if(a != 10) {
                  System.out.println("True");
            }
            else {
                  System.out.println("False");
            }

            //    TRUE ^ TRUE > FALSE
                  if(a < 15 ^ b > 15) {
                        System.out.println("True");
                  }
                  else {
                        System.out.println("False");
                  }

            //    Example for more than two conditions
            //    TRUE && TRUE && TRUE > TRUE
```

```
                    if(a < 15 && b > 15 && b != 21) {
                          System.out.println("True");
                    }
                    else {
                          System.out.println("False");
                    }
          }
}
```

```
##--> 04TernaryOperator_App.java<-- ##

package com.udemy.app;

public class App {

      public static void main(String[] args) {

            /**
             * Ternary Operator
             *
             * result = testCondition ? value1 : value2
             *
             **/

            // first, basic example
            int a = 10;
            int b;
            b = a < 20 ? 22 : 33;
            System.out.println(b);

            // second, smaller of two values example
            int smallerValue;
            int c = 3;
            int d = 2;
            smallerValue = c < d ? c : d;
            System.out.println("Smaller value = " + smallerValue);

            /**
             * Ternary operator with more than one condition
             *
             * result = testCondition1 ? value1 : testCondition2 ? value2
: value3
             *
             **/

            // example
            int e = 9; // int e = 0
            System.out.println(e == 0 ? "zero" : e < 0 ? "negative
number" : "positive number");

            /**
             * The example above is appropriate, but keep in mind: Do not
use too long and complex chained ternary
             * statements. Code readability is important!
             *
             **/
      }
}
```

```
##--> 05TypeComparisonOperator_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Type Comparison Operator is also known as the instanceof
operator
       *
       * The instanceof operator compares an object to a specified type
and returns true if the type of object and
       * the specified type are the same.
       *
       * if (objectReference instanceof type)
       *
       **/

      public static void main(String[] args) {

            /**
             * example1
             **/
            String testSubject = "apple";

            boolean testResult = testSubject instanceof String;

            System.out.println("The result of the test example1: " +
testResult);

            /**
             * example2
             **/

            ChildClass child = new ChildClass();

            boolean testResult2 = child instanceof App;

            System.out.println("The result of the test example2: " +
testResult2);
      }
}
```

```
##--> 05TypeComparisonOperator_ChildClass.java<-- ##

package com.udemy.app;

public class ChildClass extends App{
            //
}
```

```
##--> 07AutoboxingUnboxing_App.java<-- ##

package com.udemy.app;


public class App {

      /**
       *          Autoboxing - Unboxing
       **/

      public static void main(String[] args) {

            /**
             * Autoboxing, primitive -> Object
             */

            Integer intObjOldPractice = new Integer(100);  //old
practice, don't use it

            /**
             * Number, Boolean, Character constructors, e.g.
Integer(int), Boolean(boolean), Character(char)
             * are deprecated (not recommended) since version Java 9
             *
             * Oracle says: "The static factory valueOf(int) is generally
a better choice,
             * as it is likely to yield significantly better space and
time performance."
             */

            Integer intObj = Integer.valueOf(200);

            //without valueOf (valueOf working in the background) same
result
            Integer intObj2 = 200;

            System.out.println(intObj);

            /**
             * Unboxing, Object -> primitive
             */

            int intPrimitive = intObj;

            System.out.println(intPrimitive == intObj);
            System.out.println(intPrimitive);

            /**
             * Don't forget that behind the scenes a method is getting
called every time when Autoboxing, or Unboxing happen.
             * This has a bad effect for performance, especially if many
of these happen.
```

```
            * So autoboxing and unboxing are helpfull, but use them only
if you need it, otherwise prefer primitives.
            **/
      }
}
```

```
##--> 08WrapperClassesUsefulMethods_App.java<-- ##

package com.udemy.app;


public class App {

      /**
       *          Useful methods of the Wrapper Classes
       *      (Number classes, Boolean Class, Character Class)
       *
       **/

      public static void main(String[] args) {

            /**
             *    valueOf() method
             *          We can use this method to create Wrapper object
for given primitive or String
             */

      //valueOf(String s)
            Integer IntObj = Integer.valueOf("20");
            Double DoubObj = Double.valueOf("20.0");
            Boolean BoolObj = Boolean.valueOf("false");
            //warning! if you give wrong value Java throws a
RuntimeException (NumberFormatException)
            //Integer I2 = Integer.valueOf("twenty");
            System.out.println("Test valueOf(String s) method: "+ IntObj
+", "+ DoubObj +", "+ BoolObj);

      //valueOf(String s, int radix)
            Integer IR = Integer.valueOf("11010",2);  //in this example:
binary system (The range of the radix is 2 to 36.)
            System.out.println("Test valueOf(String s, int radix) method:
"+ IR);

      //valueOf(primitive p);
            Integer IP = Integer.valueOf(30);
            Double DP = Double.valueOf(30.33);
        Character CP = Character.valueOf('a');
        System.out.println("Test valueOf(primitive p) method: "+ IP +",
"+ DP +", "+ CP);

            /**
             *    xxxValue() method
             *          The method converts the value of the Number Object
that invokes the method
             *          to the primitive data type that is returned from
the method.
             */

        System.out.println("Test xxxValue() methods:");
        System.out.println(IntObj.byteValue());
```

```
        System.out.println(IntObj.shortValue());
        System.out.println(IntObj.intValue());
        System.out.println(IntObj.longValue());
        System.out.println(IntObj.floatValue());
        System.out.println(IntObj.doubleValue());

           /**
            *    parseXxx() method
            *         We can use this method to convert String to
primitive.
            */

    //parseXxx(String s)
        int i = Integer.parseInt("40");
        double d = Double.parseDouble("40.44");
        boolean b = Boolean.parseBoolean("true");
        System.out.println("Test parseXxx(String s) method: "+ i +", "+ d
+", "+ b);

    //parseXxx(String s, int radix)
        //this method convert specified radix String to primitive
        int i2 = Integer.parseInt("11010", 2);
        long l = Long.parseLong("1111", 2);
        System.out.println("Test parseXxx(String s, int radix) method: "+
i2 + ", "+ l);

           /**
            *    toString() method
            *         This method converts Wrapper object or primitive
to String.
            */

     //toString()
        String s = IntObj.toString();
        System.out.println("Test toString() method: "+ s);

     //toString(primitive p)
        String s2 = Integer.toString(150);
        String s3 = Character.toString('b');
        System.out.println("Test toString(primitive p) method: "+ s2 + ",
"+ s3);

     //toString(primitive p, int radix);
        String s4 = Long.toString(13,2);
        System.out.println("Test toString(primitive p, int radix) method:
"+ s4);

      }
}
```

```
##--> 09NumberAndBooleanClassesMethods_App.java<-- ##

package com.udemy.app;


public class App {

      /**
       *          Number & Boolean Classes Useful Methods
       *
       **/

      public static void main(String[] args) {


      //comperTo()
      //This method is used to compare the specified argument and the
number object, but two different types cannot be compared.
            Integer I = 10;
            System.out.println("Test Integer.comperTo():" +
I.compareTo(9));  //Integer is greater than the argument: 1
            System.out.println("Test Integer.comperTo():" +
I.compareTo(10)); //Integer is equal to the argument: 0
            System.out.println("Test Integer.comperTo():" +
I.compareTo(11));  //Integer is less than the argument: -1

            Boolean B = true;
            Boolean B2 = false;
            Boolean B3 = true;
            System.out.println("Test Boolean.comperTo():"+
B.compareTo(B2)); //Boolean represents true and the argument represents
false: 1
            System.out.println("Test Boolean.comperTo():"+
B.compareTo(B3)); //Boolean represents the same value as the argument: 0
            System.out.println("Test Boolean.comperTo():"+
B2.compareTo(B)); //Boolean represents false and the argument represents
true: -1

      //equals()
      //The method determines whether the Number object that invokes the
method is equal to the object that is passed as an argument.
            Integer I2 = 10;
            Integer I3 = 9;
            System.out.println("Test Integer.equals():" + I.equals(I2));
            System.out.println("Test Integer.equals():" + I.equals(I3));

            System.out.println("Test Boolean.equals():" + B.equals(B3));
            System.out.println("Test Boolean.equals():" + B.equals(B2));

            //The difference between the == operator and the equals
method will be discusses in the later (String) lectures

      //ceil() & floor()
            double d = 9.51;
```

```
            System.out.println("Test ceil():" + Math.ceil(d));
            System.out.println("Test floor():" + Math.floor(d));

      //round()
            double d2 = 20.59;
            System.out.println("Test round():" + Math.round(d2));

      //min() & max()
            System.out.println("Test min():" + Math.min(d, d2));
            System.out.println("Test max():" + Math.max(d, d2));

      //random()   This method returns a double between 0.0 and 1.0
            System.out.println("Test random():" +  Math.random() );

      //You can find other methods of Number & Boolean Classes in the
Java API documentation.

      }
}
```

```
##--> 10CharacterClassMethods_App.java<-- ##

package com.udemy.app;


public class App {

      /**
       *          Character Class Useful Methods & Escape Sequences
       *
       **/

      public static void main(String[] args) {

      //isLetter() & isDigit()
            Character ch = 'a';
            Character ch2 = '9';
            System.out.println("Test isLetter() method: "+
Character.isLetter(ch));
            System.out.println("Test isLetter() method: "+
Character.isLetter(ch2));
            System.out.println("Test isDigit() method: "+
Character.isDigit(ch));
            System.out.println("Test isDigit() method: "+
Character.isDigit(ch2));

      //isWhitespace()
            Character ch3 = ' ';
            System.out.println("Test isWhitespace() method: "+
Character.isWhitespace(ch3));

      //isUpperCase() & isLowerCase()
            Character ch4 = 'A';
            System.out.println("Test isUpperCase() method: "+
Character.isUpperCase(ch4));
            System.out.println("Test isLowerCase() method: "+
Character.isLowerCase(ch4));

      //toUpperCase & toLowerCase
            Character ch5 = 'e';
            System.out.println("Test toUpperCase() method: "+
Character.toUpperCase(ch5));
            System.out.println("Test toLowerCase() method: "+
Character.toLowerCase(ch4));

      //Escape Sequences
            //System.out.println("Java said "Hello!" to the world.");
//example for the problem = Syntax Error
            System.out.println("Test Escape Sequences:");

            //          \"
            System.out.println("Java said \"Hello!\" to the world.");
//Insert a double quote character in the text at this point.
            //          \'
```

```
            System.out.println("Java said \'Hello!\' to the world.");
//Insert a single quote character in the text at this point.
            //          \\
            System.out.println("Java said \\Hello!\\ to the world.");
//Insert a backslash character in the text at this point.
            //          \t
            System.out.println("Java said \tHello! to the world.");
//Insert a tab in the text at this point.
            //          \n
            System.out.println("Java said \nHello! to the world.");
//Insert a newline in the text at this point.


      //You can find other methods of Character Class in the Java API
documentation.

      }
}
```

```
##--> 12GarbageCollection_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Requesting JVM for running Garbage Collector - There is no
guarantee that GC will run!!! It only suggests
       * to the JVM that now might be a good time for some garbage
collection. The call System.gc() is effectively
       * equivalent to the call : Runtime.getRuntime().gc()
       *
       * Nullifying the reference variable: left the previous object
without reference -> Elligible for garbage
       * collection
       *
       **/

      public static void main(String[] args) {

            // example 1
            Car a = new Car();
            a = null; // Nullifying the reference variable
            System.gc(); // Call GC
            /**
             * Nothing happen when we call System.gc() because there is
only 1 object elligible for garbage collection and
             * the JVM doesn't feel the need for collect garbage
immediately despite of the command System.gc()
             */

            // example 2 (50 objects with System.gc() )
//          for (int i = 0; i < 50; i++) {
//                // Create a new Car object
//                Car a2 = new Car();
//                a2 = null;
//                System.gc();
//          }

            // example 3 (500000 objects without System.gc())
//          for (int i = 0; i < 500000; i++) {
//                // Create a new Car object
//                Car a3 = new Car();
//                a3 = null;
//                //System.gc();
//          }

            /**
             * To call System.gc() is a bad practice. Let's all recognize
that System.gc() isn't an absolute solution and
             * you can't rely on it as a fix and it's probably a sign
that you have some less-than-good code on your
```

```
            * hands. Alternatives instead of System.gc(): - Re-read
code. You might be holding on to references to
            * objects without realizing it. - Adjust your JVM memory
arguments and run profiling tools to figure out
            * what's going on and why you seem to be running out of
memory.
            */
      }
}
```

```
##--> 12GarbageCollection_Car.java<-- ##

package com.udemy.app;

public class Car {

      /**
       * Just before destroying an object, Garbage Collector calls
finalize() method on the object to perform
       * cleanup activities. Once finalize() method completes, Garbage
Collector destroys that object.
       *
       * finalize() is deprecated from JDK 9 : The finalization mechanism
is inherently problematic. Finalization
       * can lead to performance issues, deadlocks, and hangs. -> don't
use it
       **/
      @Override
      protected void finalize() throws Throwable {
            System.out.println(this + " Object cleaned from memory");
      }
}
```

```
##--> 14Heap_App.java<-- ##

package com.udemy.app;

/**
 *
 * Heap Memory in practice
 *
 */

public class App {

      public static void main(String[] args) {
            /**
             * Every Java application has a single instance of class
Runtime that allows the application to interface with
             * the environment in which the application is running. The
current runtime can be obtained from the
             * getRuntime method.
             */

            Runtime runtime = Runtime.getRuntime(); // Get an instance of
the Runtime class

            /**
             * totalMemory: Memory currently available for Heap (in
Bytes), total amount of memory in the JVM the default
             * value is chosen at runtime based on system configuration
             */

            // long totalMemory = runtime.totalMemory();
            // System.out.println("Heap memory currently available for
use: " + totalMemory + " Bytes");

            /**
             * 1 MB = 1024 KB, 1 KB = 1024 Bytes -> 1MB = 1024 * 1024
Bytes -> 1MB = 1.048.576 Bytes
             */
            long MegaBytes = 1048576;
            long totalMemory = runtime.totalMemory() / MegaBytes;
            System.out.println("Heap memory currently available for use:
" + totalMemory + " MB");

            /**
             * maxMemory: Returns the maximum amount of memory that the
JVM will attempt to use. JVM can expand Heap
             * whenever it needs, but it cannot grow beyond this size.
             */

            long maxMemory = runtime.maxMemory() / MegaBytes;
            System.out.println("Maximum memory Heap can use: " +
maxMemory + " MB");

            /**
```

```
            * freeMemory: Returns the amount of free memory in the Java
Virtual Machine. Free memory still available.
            */

           long freeMemory = runtime.freeMemory() / MegaBytes;
           System.out.println("Free memory still available in Heap: " +
freeMemory + " MB");

           /**
            * Memory currently used by Heap
            */

           long memoryInUse = totalMemory - freeMemory;
           System.out.println("Memory currently used by Heap: " +
memoryInUse + " MB");

      }
}
```

```
##--> 16ConstructorOverloading_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Constructor Overloading with Constructor Chaining
       **/

      public static void main(String[] args) {

            House house = new House(2, 3, 6);
            house.print();

            House house2 = new House(2, 4);
            house2.print();

            House house3 = new House();
            house3.print();
      }
}
```

```
##--> 16ConstructorOverloading_House.java<-- ##

package com.udemy.app;

public class House {

	private int numDoors;
	private int numRooms;
	private int numWindows;

	public House() {
		this(1); // calls constructor with 1 parameters (Constructor
Chaining)
	}

	public House(int numDoors) {
		this(numDoors, 5); // calls constructor with 2 parameters
(Constructor Chaining)
	}

	public House(int numDoors, int numRooms) {
		this(numDoors, numRooms, 10); // calls constructor with 3
parameters (Constructor Chaining)
	}

	// this constructor does all the work (Constructor Chaining)
	public House(int numDoors, int numRooms, int numWindows) {
		this.numDoors = numDoors;
		this.numRooms = numRooms;
		this.numWindows = numWindows;
	}

	public void print() {
		System.out.println(
				"The house has " + numDoors + " door(s), " +
numRooms + " room(s) and " + numWindows + " window(s)");
	}

}
```

```
##--> 17MethodOverloading_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Method Overloading
       *
       * If a class has multiple methods having same name but different
in parameters, it is known as Method
       * Overloading. There are two ways to overload the method in java:
       * 1. changing number of arguments
       * 2. changing the data type
       **/

      public static void main(String[] args) {

            System.out.println(add(8, 15)); // two integer arguments
            System.out.println(add(12.3, 12.6)); // two double arguments
            System.out.println(add(9, 20, 30)); // three integer
arguments
      }

      static int add(int a, int b) {
            System.out.println("adding 2 int method");
            return a + b;
      }

      static double add(double a, double b) {
            System.out.println("adding 2 double method");
            return a + b;
      }

      static int add(int a, int b, int c) {
            System.out.println("adding 3 int method");
            return a + b + c;
      }
}
```

```
##--> 20StringComparison_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Strings comparison
       *
       * Difference between == and .equals() method both equals() method
and "==" operator are used to compare
       * objects to check equality, but there are differences between
them:
       *
       * == tests for reference equality (whether they are pointing to
the same object). equals() it's a comparison
       * of values in the objects (tests for value equality, whether they
are logically "equal").
       *
       * If a class does not override the equals method, then by default
it uses equals(Object o) method of the
       * closest parent class that has overridden this method.
       *
       **/

      public static void main(String[] args) {

            // let's see our example from the previous lecture, and test
comparisons:
            String n1 = "Kevin";
            String n2 = "Kevin";
            String n3 = new String("Kevin");
            String n4 = new String("Kevin");
            String n5 = new String("Peter");

            System.out.print("n1 == n2: ");
            System.out.println(n1 == n2); // true
            System.out.print("n1.equals(n2): ");
            System.out.println(n1.equals(n2)); // true

            System.out.print("n2 == n3: ");
            System.out.println(n2 == n3); // false
            System.out.print("n2.equals(n3): ");
            System.out.println(n2.equals(n3)); // true

            System.out.print("n3 == n4: ");
            System.out.println(n3 == n4); // false
            System.out.print("n3.equals(n4): ");
            System.out.println(n3.equals(n4)); // true

            System.out.print("n4 == n5: ");
            System.out.println(n4 == n5); // false
            System.out.print("n4.equals(n5): ");
            System.out.println(n4.equals(n5)); // false
```

```
            // use equals() if you want to test whether two strings have
the same value or
            // not

      }
}
```

```
##--> 21StringIntern_App.java<-- ##

package com.udemy.app;


public class App {

      /**
       *          Strings intern() method
       *
       **/

      public static void main(String[] args) {

            /**
             * intern() method
             * Returns a canonical representation for the string object.
Intern() method saves memory from a program.
             *
             * When the intern method is invoked, if the pool already
contains a string equal to this String object,
             *  then the string from the pool is returned. Otherwise,
this String object is added to the pool and a
             *  reference to this String object is returned.
             *
             * Keep in mind, that Java automatically interns String
literals.
             *          (So in the background String = "abc"; is equal
String = "abc".intern;)
             *
             * This means that the intern() method is to be used on
Strings that are constructed with new String().
             */


            //intern example
            String constantString = "Kevin";

            String newString = new String("Kevin");
            System.out.println(constantString==newString); //false
            System.out.println(constantString.equals(newString)); //true

            newString = newString.intern();
            System.out.println(constantString==newString); //true
            System.out.println(constantString.equals(newString));
//true!!!

            /**
             * What happend in the background?
             *   before: newString was pointing to the Heap to an object
which value was "Kevin"
             *   after intern:  newString is pointing to the String
Constant Pool to an object which value is "Kevin"
```

```
            *   so after intern constantString and newString are
pointing to the same object in the Pool
            */

      }
}
```

```
##--> 23StringMethods_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;

public class App {

      /**
       * String methods
       **/

      public static void main(String[] args) {

            /**
             * previously:
             *
             * - length() method: returns the length of this string.
e.g.: str1.length()
             * - equals(Object anObject) method: a comparison of values
in the objects. e.g.: str1.equals(str2)
             * - intern() method: returns a canonical representation for
the string object. e.g.: str1.intern();
             */

            /**
             * concat(String str) method
             * As we have seen in the previous chapter this returns a new
string that is string1 with string2 added to it
             * at the end.
             *
             * + operator
             * Strings are more commonly concatenated with the +
operator, and it brings the same results.
             */

            String string1 = "Hello ";
            String string2 = "Java!";
            String string3 = string1.concat(string2);
            String string4 = string1 + string2;
            System.out.println(string3); // System.out.println(string4);
            System.out.println(string3.equals(string4));

            /**
             * charAt(int index) method
             * Returns the specified character from the string.
             */

            System.out.println("Char in search position: " +
string1.charAt(1)); // "Hello " (1) -> e

            /**
             * indexOf(String str)
```

```
            * Returns the index within this string of the first
occurrence of the specified substring.
            */

           System.out.println("Found index is: " +
string1.indexOf("llo")); // "Hello " -> "llo" -> 2

           /**
            * substring(int beginIndex) method
            * Returns a new string that is a substring of this string.
            */

           String str = "Hello Java!";
           System.out.println(str.substring(6));

           /**
            * replace(char oldChar, char newChar)
            * Returns a new string resulting from replacing all
occurrences of oldChar in this string with newChar.
            *
            * replaceAll(String regex, String replacement)
            * Replaces each substring of this string that matches the
given regular expression with the given
            * replacement.
            */

           System.out.println(str.replace('J', 'L'));
           System.out.println(str.replaceAll("Java", "Peter"));

           /**
            * trim()
            * Returns a copy of the string, with leading and trailing
whitespace omitted.
            */

           String s = "    trim is a useful method in Java    ";
           System.out.println(s + ".");
           System.out.println(s.trim());

           /**
            * split(String regex, int limit)
            * Splits this string around matches of the given regular
expression.
            *
            * regex: regular expression, limit: how many strings to be
returned
            */

           String strSplit = "string&split&method";
           String[] arrayOfStrings = strSplit.split("&", 3);

           for (String strSp : arrayOfStrings)
                 System.out.println(strSp);
```

```
            /**
             * You can find other methods of String Class in the Java API
documentation.
             */

      }
}
```

```
##--> 25StringSBuilderSBuffer_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;

public class App {

      /**
       * String VS String Builder VS String Buffer
       **/

      public static void main(String[] args) {

            /**
             * String VS String Builder object generation / hashCode()
             */

            String string = "Alexander";
            System.out.println(string.hashCode());
            string = string + " the Great";
            System.out.println(string.hashCode());
            System.out.println(string);

            StringBuilder sb = new StringBuilder("Alexander");
            System.out.println(sb.hashCode());
            sb.append(" the Great");
            System.out.println(sb.hashCode());
            System.out.println(sb);

            /**
             * HashCode() method is used to generate a numerical
representation of the contents of an object. By default
             * the hashCode() returns an integer that represents the
internal memory address of the object. Whenever it is
             * invoked on the same object more than once during an
execution of a Java application, the hashCode method
             * must consistently return the same integer.
             */

            /**
             * String Buffer VS String Builder speed comparison
             */

            // String Buffer
            StringBuffer stringbuffer = new StringBuffer();
            long startTime = System.currentTimeMillis();

            for (int i = 0; i < 10000000; i++) {
                  stringbuffer.append("Java");
            }

            long endTime = System.currentTimeMillis();
            long duration = endTime - startTime;
```

```
            System.out.println("Duration of StringBuffer's running is: "
+ duration + " milliSecond");

            // String Builder
            StringBuilder stringbuilder = new StringBuilder();
            long startTime2 = System.currentTimeMillis();

            for (int i = 0; i < 10000000; i++) {
                  stringbuilder.append("Java");
            }

            long endTime2 = System.currentTimeMillis();
            long duration2 = endTime2 - startTime2;
            System.out.println("Duration of StringBuilder's running is: "
+ duration2 + " milliSecond");

      }
}
```

```
##--> 26StringBuilderMethods_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;

public class App {

      /**
       * String Builder Methods (same methods are in String Buffer class)
       **/

      public static void main(String[] args) {

            /**
             * charAt(), indexOf(), length(), and substring() These
methods work exactly the same as in the String class.
             */

            /**
             * append()
             * This method is by far the most frequently used method in
StringBuilder. Let's see method chaining with
             * append!
             */

            StringBuilder strBuilder = new StringBuilder("Wall Street ");
            strBuilder = strBuilder.append(12).append(", ").append("New
York");
            System.out.println(strBuilder);

            /**
             * insert()
             * This method adds characters to the StringBuilder at the
requested index and returns a reference to the
             * current StringBuilder.
             */

            System.out.println(strBuilder.insert(15, " 5th floor,"));

            /**
             * delete()
             * This method removes the characters in a substring of this
sequence.
             */

            System.out.println(strBuilder.delete(15, 26));

            /**
             * reverse()
             * This method causes this character sequence to be replaced
by the reverse of the sequence.
             */
```

```
            System.out.println(strBuilder.reverse());

            /**
             * toString()
             * Often StringBuilder is used internally for performance
purposes but the end result needs to be a String.
             * For example, maybe it needs to be passed to another method
that is expecting a String.
             */

            String s = strBuilder.toString();

            /**
             * You can find other methods of String Class in the Java API
documentation.
             */
      }
}
```

```
##--> 28Enum_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Enum
       **/

      // You can also have an Enum inside a class
      enum Seasons {
            SPRING, SUMMER, AUTUMN, WINTER
      }

      public static void main(String[] args) {

            System.out.println(Day.FRIDAY);

            Seasons mySeason = Seasons.SPRING;
            System.out.println(mySeason);

            /**
             * Enums in Switch statement Enums are often used in switch
statements to check for corresponding values
             */

            talkingSeasons(mySeason);

            /**
             * Loop Through an Enum The enum type has a values() method,
which returns an array of all enum constants.
             */

            for (Day d : Day.values()) {
                  System.out.println(d);
            }

            /**
             * Other Enum methods:
             *
https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/En
um.html#method.summary
             */

      }

      public static void talkingSeasons(Seasons mySeason) {
            switch (mySeason) {
            case SPRING:
                  System.out.println("Blossom flowers.");
                  break;

            case SUMMER:
```

```
                System.out.println("It's holiday time!");
                break;

            case AUTUMN:
                System.out.println("The leaves are falling.");
                break;

            case WINTER:
                System.out.println("Build a snowman!");
                break;

            default:
                System.out.println("If there will be a fifth season,
then I'll come here. For safety's sake. :) ");
                break;
            }
            /**
             * In switch statements, enumeration constants are used
without their enumeration type name. This is due to
             * the fact the enum type is implicitly specified in the
switch expression. It is good practice to list all
             * enum constants (even if some of them don’t do anything)
and add a default branch, usually with an exception
             * (in case a new constant gets added and someone misses the
switch statement and doesn’t update it.)
             */
      }
}
```

```
##--> 28Enum_Day.java<-- ##

package com.udemy.app;

//Separate Enum
public enum Day {
      MONDAY,
      TUESDAY,
      WEDNESDAY,
      THURSDAY,
      FRIDAY,
      SATURDAY,
      SUNDAY
}
```

```
##--> 30DatesAndTimesClassic_App.java<-- ##

package com.udemy.app;

import java.text.SimpleDateFormat;
import java.util.Calendar;
import java.util.Date;
import java.util.GregorianCalendar;
import java.util.TimeZone;

public class App {

      /**
       * Dates & Times, Old-classic way
       **/

      public static void main(String[] args) {

            /**
             * Current Date and Time
             */

            Date date = new Date(); // Instantiate a Date object
            System.out.println(date.toString()); // display time and date
using toString()

            /**
             * Creating Dates and Times
             *
             * Calendar & Gregorian calendar
             *
             * getInstance() : Gets a calendar using the default time
zone and locale.
             * set() : Sets the values for the calendar fields YEAR,
MONTH, and DAY_OF_MONTH.
             * getTime() : Returns a Date object representing this
Calendar's time value
             *
             * more methods:
             * Calendar:
             *
https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/Ca
lendar.html#method.summary
             * Gregorian calendar:
             *
https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/util/Gr
egorianCalendar.html
             */

            // Creating an object representing January 1, 2019 with
Calendar
            Calendar c = Calendar.getInstance();
            c.set(2015, Calendar.JANUARY, 1); // c.get(Calendar.YEAR) ->
2015 Returns a Date object representing this
```

```
                                                                //
Calendar's time value
            Date dateCal = c.getTime();

            // Creating an object representing February 1, 2019 with
GregorianCalendar
            Calendar c2 = new GregorianCalendar(2019, Calendar.FEBRUARY,
1);
            Date dateGreg = c2.getTime();

            /**
             * Notice the problem with this old way: There wasn’t a way
to specify just a date without the time. The Date
             * class represented both the date and time whether you
wanted it to or not. Trying to create a specific date
             * required more code than it should have.
             */

            System.out.println("Created Date object with Calendar: " +
dateCal);
            System.out.println("Created Date object with
GregorianCalendar: " + dateGreg);

            /**
             * Date and Time Formatting
             *
             * SimpleDateFormat: Formatter for date/time.
             * date and time patterns: E:Day name in week, y:Year,
M:Month in year, d:Day in month, h:Hour in am/pm (1-12),
             * m: Minute in hour, s:Second in minute, a:Am/pm marker,
z:Time zone, etc.
             *
             * all patterns:
https://docs.oracle.com/javase/7/docs/api/java/text/SimpleDateFormat.html
             */

            Date dateNow = new Date();
            SimpleDateFormat sdft = new SimpleDateFormat("E yyyy.MM.dd
'at' hh:mm:ss a z"); // Pattern Example
            System.out.println("Current Date: " + sdft.format(dateNow));

            /**
             * Manipulating Dates
             */

            Calendar cal = Calendar.getInstance();
            cal.set(2020, Calendar.OCTOBER, 1);
            Date mDate = cal.getTime();
            System.out.println("before manipulation: " + mDate);
            cal.add(Calendar.DATE, 2);
            mDate = cal.getTime();
            System.out.println("after manipulation: " + mDate);

            /**
```

```
            * Compare Dates
            */
           Date dateCurrent = new Date();
           System.out.println(mDate.compareTo(dateCurrent)); // 0 if
both dates are equal, Date is after/before the date

                 // argument: -1/1
           System.out.println(mDate.before(dateCurrent)); //
before/after true/false

           /**
            * TimeZone This class represents a time zone offset
            */

           TimeZone zone = TimeZone.getDefault();
           String zoneName = zone.getDisplayName();
           System.out.println("Display name for local time zone: " +
zoneName);

      }
}
```

```
##--> 31DatesAndTimesNew_App.java<-- ##

package com.udemy.app;

import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.Month;
import java.time.Period;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Date;

public class App {

      /**
       * Dates & Times - New way with java.time package (part 1)
       **/

      public static void main(String[] args) {

            /**
             * Current Date and Time
             */

            System.out.println("Current Date: " + LocalDate.now());
            System.out.println("Current Time: " + LocalTime.now());
            System.out.println("Current Date & Time: " +
LocalDateTime.now());

            /**
             * Creating Dates and Times
             */

            // date
            LocalDate dateA = LocalDate.of(2020, Month.MARCH, 15); //
Both of these examples create the same date
            LocalDate dateB = LocalDate.of(2020, 3, 15);
            // LocalDate dateC = LocalDate.of(2015, Month.MARCH, 32); //
throws DateTimeException
            System.out.println("dateA: " + dateA + ", dateA: " + dateB);

            // time
            LocalTime time = LocalTime.of(9, 33); // or (9, 33, 10), or
(9, 33, 10, 300) = (hour, minute, seconds,
                                                                    //
nanoseconds)
            System.out.println("time: " + time);

            // date & time
```

```
            LocalDateTime dateTime = LocalDateTime.of(2020, Month.MARCH,
15, 9, 33);
            // (year, month, dayOfMonth, hour, minute) but you can add
seconds, nanoseconds at the ends if you need
            LocalDateTime dateTime2 = LocalDateTime.of(dateA, time); //
this is the same
            System.out.println("dateTime: " + dateTime);
            System.out.println("dateTime2: " + dateTime2);

            /**
             * Date and Time Formatting
             *
             * DateTimeFormatter: Formatter for printing and parsing
date-time objects. ofLocalizedDate: This returns a
             * formatter that will format or parse a date.The exact
format pattern used varies by locale.
             *
             *
             * ofLocalized methods
             *                                          localDate
                   localDateTime                localTime
             * ofLocalizedDat            shows whole object
shows just date part   Throws runtime exception
             * ofLocalizedDateTime   Throws runtime exception    shows
whole object     Throws runtime exception
             * ofLocalizedTime            Throws runtime exception
shows just time part      shows whole object
             */

            LocalDate date1 = LocalDate.of(2020, Month.JANUARY, 20);
            LocalTime time1 = LocalTime.of(11, 12, 34);
            LocalDateTime dateTime1 = LocalDateTime.of(date1, time1);

            DateTimeFormatter shortDate =
DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT); // FULL, LONG,
MEDIUM, SHORT
            DateTimeFormatter shortTime =
DateTimeFormatter.ofLocalizedTime(FormatStyle.SHORT);
            DateTimeFormatter shortDateTime =
DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT,
FormatStyle.SHORT);

            System.out.println("shortDate: " + date1.format(shortDate));
// format: Formats this date-time using the

                                    // specified formatter.
            System.out.println("shortTime: " + time1.format(shortTime));
            System.out.println("shortDateTime: " +
dateTime1.format(shortDateTime));
            // System.out.println(time.format(shortDateTime));
//UnsupportedTemporalTypeException

            /**
             * Manipulating Dates and Times
```

```
            */

            LocalDate mDate = LocalDate.of(2021, Month.JULY, 04);
            System.out.println(mDate);
            mDate = mDate.plusDays(12); // plus, minus, Days, Weeks,
Months, Years, etc.
            System.out.println(mDate);
            mDate = mDate.withYear(2019); // Year, Month, etc.
            System.out.println(mDate);

            LocalDate m2Date = LocalDate.of(2020, Month.DECEMBER, 01);
            LocalTime m2Time = LocalTime.of(11, 18);
            System.out.println("Before: " + LocalDateTime.of(m2Date,
m2Time));
            LocalDateTime m2DateTime = LocalDateTime.of(m2Date,
m2Time).minusDays(1).minusHours(11); // date and time


      // methods chained
            System.out.println("After: " + m2DateTime);
      }
}
```

```
##--> 32DatesAndTimesNew2_App.java<-- ##

package com.udemy.app;

import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.Month;
import java.time.Period;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Date;

public class App {

      /**
       * Dates & Times - New way with java.time package (part 2)
       **/

      public static void main(String[] args) {

            /**
             * Compare Dates and Times In Java 8, you can use the new
isBefore(), isAfter(), isEqual() and compareTo() to
             * compare LocalDate, LocalTime and LocalDateTime.
             */

            LocalDate compDate = LocalDate.of(2021, Month.JULY, 04);
            LocalDate comp2Date = LocalDate.of(2020, Month.DECEMBER, 01);
            System.out.println(compDate.compareTo(comp2Date)); // 0 if
both dates are equal, Date is after/before the date

                  // argument: -1/1
            System.out.println(compDate.isBefore(comp2Date)); //
isBefore/isAfter true/false
            System.out.println(compDate.isEqual(comp2Date)); //
true/false

            /**
             * TimeZone Zoned date/time API is to be used when time zone
is to be considered.
             */

            ZonedDateTime dateZoned = ZonedDateTime.parse("2007-12-
03T10:15:30+01:00[Europe/Paris]");
            // The string must represent a valid date-time and is parsed
using
            // java.time.format.DateTimeFormatter.ISO_ZONED_DATE_TIME.
            System.out.println("dateZoned: " + dateZoned);
```

```
            ZoneId id = ZoneId.of("Europe/Paris");
            System.out.println("ZoneId: " + id);

            ZoneId currZone = ZoneId.systemDefault();
            System.out.println("CurrentZone: " + currZone);

            /**
             * Duration Measures time in seconds and nanoseconds. It
deals with time based amount of time. methods:
             *
https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/time/Du
ration.html#method.summary
             *
             * Period Measures time in years, months and days. It deals
with date based amount of time. methods:
             *
https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/time/Pe
riod.html#method.summary
             */

            // Duration
            LocalDateTime oldDate = LocalDateTime.of(2020, Month.AUGUST,
31, 10, 20, 55);
            LocalDateTime newDate = LocalDateTime.of(2020,
Month.SEPTEMBER, 1, 10, 20, 55);
            Duration duration = Duration.between(oldDate, newDate);
            System.out.println("Duration: " + duration);
            System.out.println("Duration: " + duration.toSeconds() + "
seconds");
            System.out.println("Duration: " + duration.toHours() + "
hours");

            // Period
            LocalDate today = LocalDate.now();
            LocalDate birthdayOfTom = LocalDate.of(1990, Month.APRIL,
22);
            Period p = Period.between(birthdayOfTom, today);
            System.out.println("Period: " + p);
            System.out.println(
                        "Tom is " + p.getYears() + " years, " +
p.getMonths() + " months, and " + p.getDays() + " days old.");

            /**
             * Backward Compatibility A toInstant() method is added to
the original Date and Calendar objects. It can be
             * used to convert them to the new Date/Time API.
             */

            Date currentDate = new Date();
            System.out.println("Current date: " + currentDate);

            Instant now = currentDate.toInstant(); // Get the instant of
current date in terms of milliseconds
            ZoneId currentZone = ZoneId.systemDefault();
```

```
            LocalDateTime localDateTime = LocalDateTime.ofInstant(now,
currentZone);
            System.out.println("Local date: " + localDateTime);

            ZonedDateTime zonedDateTime = ZonedDateTime.ofInstant(now,
currentZone);
            System.out.println("Zoned date: " + zonedDateTime);
      }
}
```

```
##--> 34NestedIf_App.java<-- ##

package com.udemy.app;

import java.lang.reflect.Array;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.Month;
import java.time.Period;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Date;
import java.util.List;

public class App {

      /**
       * Nested if statement
       */

      public static void main(String[] args) {

            // nested if example
            int number1 = 2;
            int number2 = 87;
            int number3 = 100;
            int greatestNumber;

            if (number1 >= number2) {
                  if (number1 >= number3) {
                        greatestNumber = number1;
                  } else {
                        greatestNumber = number3;
                  }
            } else {
                  if (number2 >= number3) {
                        greatestNumber = number2;
                  } else {
                        greatestNumber = number3;
                  }
            }

            System.out.println("The highest number in the list is " +
highestNumber);

      }
```

}

```
##--> 36NestedLoops_App.java<-- ##

package com.udemy.app;

import java.lang.reflect.Array;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.Month;
import java.time.Period;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Date;
import java.util.List;

public class App {

      /**
       * Nested loops & Optional Labels
       */

      public static void main(String[] args) {

            // Nested loop example
            for (int i = 1; i <= 3; ++i) {

                  System.out.println("Outer loop iteration " + i);

                  for (int j = 1; j <= 2; ++j) {
                        System.out.println("i = " + i + "; j = " + j);
                  }
            }

            // Nested loop with Optional Labels usage example
            System.out.println(""); // separation line

            OUTER_LOOP: for (int i = 1; i <= 3; ++i) {

                  System.out.println("Outer loop iteration " + i);

                  INNER_LOOP: for (int j = 1; j <= 2; ++j) {
                        System.out.println("i = " + i + "; j = " + j);
                        if (j == 2 && i == 2) {
                              break OUTER_LOOP;
                              /**
                               * Without label reference the break
statement default behavior is to jump out from the inner loop
```

```
                                * only. The optional label parameter allows
us to break out of a higher level outer loop.
                                */
                        }
                    }
            }
        }
}
```

```
##--> 38ArraysMethods_App.java<-- ##

package com.udemy.app;

import java.lang.reflect.Array;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.Month;
import java.time.Period;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Arrays;
import java.util.Collections;
import java.util.Date;

public class App {

      /**
       * Arrays methods
       *
       * The toString() method returns a string representation of the
contents of the specified array.
       *
       * The sort() method sorts the integers in an ascending order.
       *
       * The binarySearch() method searches the specified array of ints
for the specified value The array must be
       * sorted prior to making this call. If it is not sorted, the
results are undefined. If the array contains
       * multiple elements with the specified value, there is no
guarantee which one will be found. Return: index of
       * the search key, if it is contained in the array; otherwise, (-
(insertion point) - 1). The insertion point
       * is defined as the point at which the key would be inserted into
the array: the index of the first element
       * greater than the key, or array.length if all elements in the
array are less than the specified key. Note
       * that this guarantees that the return value will be >= 0 if and
only if the key is found.
       *
       * The fill() method assigns the specified int value to each
element of the specified array of ints.
       *
       * The copyOf() method copies the specified number of elements to a
new array.
       *
       * //The equals() method compares the two arrays. Two arrays are
equal if they contain the same elements in
       * the same order.
```

```
       **/

      public static void main(String[] args) {

            int[] numbers = { 10, 5, 20, 33, 11 };
            System.out.println("Original array: " +
Arrays.toString(numbers)); // toString() method !!!

            Arrays.sort(numbers); // sort() method !!!
            System.out.println("Sorted array:   " +
Arrays.toString(numbers));

            int searchedElement = 33;
            int result = Arrays.binarySearch(numbers, searchedElement);
// binarySearch() method !!!
            if (result >= 0) {
                  System.out.println("The searched element is in the
array, at the position: " + result);
            } else {
                  System.out.println("The searched element is not in the
array");
            }

            Arrays.fill(numbers, 77); // fill() method !!!
            System.out.println(Arrays.toString(numbers));

            int[] numbersB = Arrays.copyOf(numbers, 5); // copyOf()
method !!!

            if (Arrays.equals(numbers, numbersB)) { // equals() method
!!!
                  System.out.println("The two array is equal");
            } else {
                  System.out.println("The two array is not equal");
            }

            System.out.println(Arrays.toString(numbersB));
      }
}
```

```
##--> 39PassingReturningArrays_App.java<-- ##

package com.udemy.app;

import java.lang.reflect.Array;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.Month;
import java.time.Period;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Arrays;
import java.util.Collections;
import java.util.Date;

public class App {

      /**
       * Passing Arrays to Method - Returning Array from a Method &
Anonymous Array
       *
       * Passing Arrays to Methods You can pass arrays to a method just
like normal variables. When we pass an array
       * to a method as an argument, actually the address of the array in
the memory is passed (reference).
       * Therefore, any changes to this array in the method will affect
the array.
       *
       * Anonymous Array An array in Java without any name is anonymous
array. It is an array just for creating and
       * using instantly. It can be used only once. Anonymous array is
passed as an argument of method. Anonymous
       * array is created and initialized in the same line.
       **/

      public static void main(String[] args) {

            /**
             * Passing Arrays to Method - Returning Array from a Method &
Anonymous Array example
             */

            int[] numbers = { 5, 7, 9 };
            System.out.println(Arrays.toString(numbers));

            int[] numbersBackwards = reverseOrder(numbers); // Passing
Arrays to the Method
            System.out.println(Arrays.toString(numbersBackwards));
```

```
            System.out.println(Arrays.toString(reverseOrder(new int[] {
1, 2, 3 }))); // Anonymous Array

      }

      private static int[] reverseOrder(int[] num) {
            int[] numbersReverseOrder = new int[num.length];

            int position = num.length - 1;

            for (Integer n : num) {
                  numbersReverseOrder[position] = n;
                  position--;
            }

            return numbersReverseOrder; // Returning Array from the
Method
      }
}
```

```
##--> 41MultidimensionalArrays_App.java<-- ##

package com.udemy.app;

import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.Month;
import java.time.Period;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Arrays;
import java.util.Collections;
import java.util.Date;

public class App {

      /**
       * Multidimensional Arrays 2D & 3D Arrays Creation and Iterating
       **/

      public static void main(String[] args) {

            /**
             * Creation and Iterating through 2D array
             */

            int[][] myNumbers = { { 1, 2, 3 }, { 4 }, { 5, 6 } };//
irregular declaration

            System.out.println("Irregular 2D Array:");
            for (int[] innerArray : myNumbers) {
                  for (int data : innerArray) {
                        System.out.print(data + ", ");
                  }
                  System.out.print("\n");
            }

            int[][] myNumbers2 = new int[3][3];// 3 row and 3 column

            System.out.println("Regular 2D Array, with auto/default zero
initialization:");
            for (int[] innerArray : myNumbers2) {
                  for (int data : innerArray) {
                        System.out.print(data + ", ");
                  }
                  System.out.print("\n");
            }

            // non-existent index danger
```

```
            // System.out.println(myNumbers[1][1]);
//ArrayIndexOutOfBoundsException
            System.out.println(myNumbers2[1][1]); // no problem because
of the auto/default initialization, it has a value
                                                                        //
(zero)

            /**
             * Creation and Iterating through 3D array Three
             */
            System.out.println(""); // separation new line

            int[][][] a3D = { { { 1, 2 }, { 3, 4 } }, { { 5, 6 }, { 7, 8
} } };

            // Basically, 3d array is an array of 2d arrays.

            System.out.println("3D Array contains:");
            for (int[][] array2D : a3D) {
                  System.out.println("inside 2D Array");
                  for (int[] array1D : array2D) {
                        for (int item : array1D) {
                              System.out.print(item + ",");
                        }
                        System.out.println("-");
                  }
            }

            /**
             * Elements in three-dimensional arrays are commonly referred
by x[i][j][k] where 'i' is the array number, 'j'
             * is the row number and 'k' is the column number
             */

            System.out.println("Accessing Elements of the 3D Array: " +
a3D[1][0][0]); // second inside 2D array/first

                                                        // row/first column:
5

      }
}
```

```
##--> 43Varargs_App.java<-- ##

package com.udemy.app;

import java.lang.reflect.Array;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.Month;
import java.time.Period;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Arrays;
import java.util.Collections;
import java.util.Date;

public class App {

      /**
       * Varargs
       **/

      public static void main(String[] args) {
            String message = "The number of int element in this calling
method: ";
            exampleMethod(message);
            exampleMethod(message, 100);
            exampleMethod(message, 100, 200, 300, 400);
      }

      //(int ... x, String s) -> compile time error;
      //(int ... x, String ... s) -> compile time error
      public static void exampleMethod(String s, int ... x) {
            System.out.println(s + x.length);
            System.out.println(Arrays.toString(x));
            }
}
```

```
##--> 45ArrayListSorting_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;
import java.util.Collections;

public class App {

      /**
       *
       * Iterating over an ArrayList & ArrayList sorting
       *
       **/

      public static void main(String[] args) {

            // loop through an ArrayList with the for-each loop
            ArrayList<String> names = new ArrayList<String>();
            names.add("Joe"); // The add() methods insert a new value in
the ArrayList.
            names.add("Kevin");
            names.add("Alan");
            names.add("James");
            for (String n : names) {
                  System.out.println(n);
            }
            System.out.println(); // separation line

            // ArrayList is a part of the Java collection framework
that's why we can use Collections.sort statement
            Collections.sort(names);
            System.out.println("Sorted names:");
            for (String n : names) {
                  System.out.println(n);
            }

      }
}
```

```
##--> 46ArrayListMethods_App.java<-- ##

package com.udemy.app;

import java.lang.reflect.Array;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.Month;
import java.time.Period;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Date;
import java.util.List;

public class App {

      /**
       * ArrayList methods
       *
       * add() : insert a new value in the ArrayList (or into a specific
position)
       * remove() : remove the first matching value in the ArrayList (or
remove the element at a specified index)
       * set() : changes one of the elements of the ArrayList without
changing the size
       * contains() : checks whether a certain value is in the ArrayList
       * size() : shows the size of the ArrayList
       * clear() : provides an easy way to discard all elements of the
ArrayList
       * isEmpty() : shows the ArrayList is empty or not
       * equals() : ArrayList has a custom implementation of equals() so
you can compare two lists to see if they
       *         contain the same elements in the same order
       **/

      public static void main(String[] args) {

            /**
             * add() method
             */
            ArrayList<String> cars = new ArrayList<String>();
            cars.add("Ford");
            cars.add("Honda");
            cars.add("Mercedes");
            cars.add("Toyota");
```

```
            showArrayList(cars);

            cars.add(2, "Kia");
            // cars.add(10, "Kia"); //if the position does not fit for
the ArrayList:
            // IndexOutOfBoundsException

            showArrayList(cars);

            /**
             * remove() method
             */
            cars.remove("Toyota");
            cars.remove(2);
            // cars.remove(10); //if the position does not exists in the
ArrayList:
            // IndexOutOfBoundsException

            showArrayList(cars);

            /**
             * set() method
             */
            cars.set(0, "Tesla");

            showArrayList(cars);

            /**
             * contains & size() methods
             */
            System.out.println(cars.contains("Ford"));
            System.out.println(cars.size());

            /**
             * clear() method
             */
            cars.clear();
            showArrayList(cars);

            /**
             * isEmpty() method
             */
            System.out.println(cars.isEmpty());

            /**
             * equals() method
             */
            ArrayList<String> cars2 = new ArrayList<String>();
            cars.add("Nissan");
            System.out.println(cars.equals(cars2));

      }

      public static void showArrayList(ArrayList<String> cars) {
```

```
            System.out.print("Current elements in the Cars ArrayList:
");
            for (String c : cars) {
                  System.out.print(c + "  ");
            }
            System.out.println(); // separation line
      }
}
```

```
##--> 48GenericsMethods_App.java<-- ##

package com.udemy.app;

import java.lang.reflect.Array;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.time.Month;
import java.time.Period;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Date;
import java.util.List;

public class App {

      /**
       * Generics methods
       *
       * 1.) method overloading VS generics 2.) Generic method with two
types
       **/

      public static void main(String[] args) {

            ArrayList<String> list = new ArrayList<>();
            list.add("Hello Java!");
            String s = list.get(0);

            Integer[] arrayInt = { 1, 2, 3 };
            Character[] arrayChar = { 'A', 'B', 'C' };

            /**
             * 1.) Old solution with method overloading with two method
VS new solution with one Generic method (callings)
             */
            printArray(arrayInt);
            System.out.println(); // new line
            printArray(arrayChar);
            System.out.println(); // new line
            /**
             * Generic method with two types (callings)
             */
            System.out.println(); // new line
            printTwoArrays(arrayInt, arrayChar);
            printTwoArrays(arrayChar, arrayInt);
```

```
            printTwoArrays(arrayInt, arrayInt);
            printTwoArrays(arrayChar, arrayChar);

      }

      /**
       * 1.) Old solution with method overloading, two method
       */
      static void printArray(Integer[] integer) {
            System.out.println("Old solution with method overloading -
method with Integer: ");
            for (Integer i : integer) {
                  System.out.print(i + ",");
            }
      }

      static void printArray(Character[] character) {
            System.out.println("Old solution with method overloading -
method with Character: ");
            for (Character c : character) {
                  System.out.print(c + ",");
            }
      }

      /**
       * 1.) Generic solution with one method
       */
//    static <T> void printArray(T[] t) {
//          System.out.println("Generic solution with one method: ");
//          for (T item : t) {
//                System.out.print(item);
//          }
//    }

      /**
       * 2.) Generic method with two types
       */
      static <T, V> void printTwoArrays(T[] t, V[] v) {
            for (T item : t) {
                  for (V item2 : v) {
                        System.out.print(item + ":" + item2 + " ");
                  }
            }
            System.out.println(); // new line
      }

}
```

```
##--> 49GenericsClass_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;

class Box<T> {
      // type T is declared
      private T element;

      // constructor
      public Box(T element) {
            this.element = element;
      }

      public T getElement() {
            return element;
      }

      public void setElement(T element) {
            this.element = element;
      }

}

public class App {

      /**
       * Generic Class example
       **/

      public static void main(String[] args) {

            // instance of Integer type
            Box<Integer> intBox = new Box<Integer>(15);
            System.out.println(intBox.getElement());

            // instance of String type
            Box<String> stringBox = new Box<String>("Hello Java!");
            System.out.println(stringBox.getElement());

      }
}
```

```
##--> 50GenericsWildcardsUpperBounded_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;
import java.util.List;

public class App {

      /**
       * Generics upper bounded wildcard example
       *
       * You can use an upper bounded wildcard to relax the restrictions
on a variable.
       *
       * In our example to write the method that works on lists of Number
and the subtypes of Number, such as
       * Integer, Double, and Float, you would specify. List<? extends
Number> list
       **/

      public static void main(String[] args) {

            List<Integer> listInteger = new ArrayList<>();
            listInteger.add(10);
            listInteger.add(20);
            process(listInteger);

            List<Double> listDouble = new ArrayList<>();
            listDouble.add(10.5);
            listDouble.add(20.3);
            process(listDouble);
      }

      /**
       * Without wildcard this looks like: List<Integer> list, or
List<Double> list but then only one specific type
       * is good for calling this method
       *
       * you can't add with upper bounded wildcard, but if you change to
lower bounded wildcard you can <? super
       * Number>
       */

      public static void process(List<? extends Number> list) { // upper
bounded wildcard
            // list.add(22); //error -> you can't add with upper bounded
wildcard
            for (Number item : list) {
                  System.out.println(item);
            }
      }
}
```

```
##--> 51GenericsWildcardsLowerBounded_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class App {

      /**
       * Generics lower bounded wildcard example
       *
       * The Upper Bounded Wildcards section shows that an upper bounded
wildcard restricts the unknown type to be a
       * specific type or a subtype of that type and is represented using
the extends keyword. In a similar way, a
       * lower bounded wildcard restricts the unknown type to be a
specific type or a super type of that type.
       *
       * In our example arguments can be Integer or superclass of
Integer(which is Number). List<? super Integer>
       * list
       **/

      public static void main(String[] args) {
            ArrayList<Integer> listInteger = new ArrayList<>();
            listInteger.add(1);
            listInteger.add(2);
            listInteger.add(3);
            addAndPrintElements(listInteger);

            ArrayList<Number> listNumber = new ArrayList<>();
            listNumber.add(10);
            listNumber.add(20);
            listNumber.add(30);
            addAndPrintElements(listNumber);
      }

      /**
       * Without wildcard this looks like: List<Integer> list, or
List<Number> list but then only one specific type
       * is good for calling this method
       *
       * you can add when using lower bounded wildcard
       *
       * we can't iterate with Integer or Number, only Object
       */
      public static void addAndPrintElements(List<? super Integer> list)
{
            list.add(100); // we can add
            list.add(200);
            list.add(300);
```

```
//          for(Integer i : list) {
//          System.out.println(i);
//          }

            System.out.println(list);
      }

}
```

```
##--> 52GenericsWildcardsUnbounded_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class App {

      /**
       * Generics unbounded wildcard example
       *
       * Unbounded wildcard type is specified using the wildcard
character (?). This is called a list of unknown
       * type. These are useful in the following cases:
       *
       * 1. When writing a method which can be employed using
functionality provided in Object class.
       *
       * 2. When the code is using methods in the generic class that
don't depend on the type parameter
       *
       * List<?> list
       **/

      public static void main(String[] args) {
            ArrayList<Integer> listInteger = new ArrayList<>();
            listInteger.add(1);
            listInteger.add(2);
            listInteger.add(3);

            ArrayList<Number> listDouble = new ArrayList<>();
            listDouble.add(10.50);
            listDouble.add(20.40);
            listDouble.add(30.33);

            ArrayList<String> listString = new ArrayList<>();
            listString.add("one");
            listString.add("two");
            listString.add("three");

            printAll(listInteger);
            printAll(listDouble);
            printAll(listString);

      }

      public static void printAll(List<?> list) {
            for (Object item : list)
                  System.out.println(item + " ");
      }

}
```

```
##--> 54Lists_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Enumeration;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.Stack;
import java.util.Vector;

public class App {

      /**
       * Lists
       *
       * List: A list is an ordered collection of elements that allows
duplicate
       * entries. Elements in a list can be accessed by an int index.
       *
       * - ArrayList - LinkedList - Vector (and Stack)
       **/

      public static void main(String[] args) {

            /**
             * ArrayList vs LinkedList main differences
             *
             * 1. AL act as a list, LL act as a list and queue both
             *
             * 2. AL use dynamic array to store the elements <> LL use
doubly linked list to
             * store the elements
             *
             * 3. Manipulation: AL slow, because if any element is
removed from the array,
             * all the bits are shifted in memory, LL faster because it
uses a doubly linked
             * list, so no bit shifting is required in memory
             *
             * 4. AL better for storing and accessing data, LL is better
for manipulating
             * data
             *
             */

            /**
             * ArrayList vs LinkedList example
             */

            List<String> arrayL = new ArrayList<>(); // creating
ArrayList
```

```
            List<String> linkedL = new LinkedList<>(); // creating
LinkedList

            // adding object in ArrayList
            arrayL.add("Kevin");
            arrayL.add("Joe");
            arrayL.add("Peter");
            // adding object in LinkedList
            linkedL.add("Kate");
            linkedL.add("Judy");
            linkedL.add("Julia");

            // iterate through with Iterator
            Iterator<String> itr = arrayL.iterator();
            System.out.println("ArrayList elements: ");
            while (itr.hasNext()) {
                  System.out.println(itr.next());
            }
            System.out.println(); // separation line

            // iterate through with for-each loop
            System.out.println("LinkedList elements: ");
            for (String s : linkedL) {
                  System.out.println(s);
            }

            /**
             * ArrayList vs Vector main differences
             *
             * Vector and an ArrayList are almost equivalent. Main
differences:
             *
             * 1. Vector is synchronized, which means only one thread at
a time can access
             * the code (thread-safe), while arrayList is not
synchronized, which means
             * multiple threads can work on arrayList at the same time.
             *
             * 2. ArrayList is faster than Vector
             *
             * 3. ArrayList and Vector both grow and shrink dynamically
but the way they
             * resize is different. ArrayList increments 50% of the
current array size if
             * the number of elements exceeds its capacity, while vector
increments 100% -
             * essentially doubling the current array size.
             *
             * 4. Vectors contains some legacy methods, for example
Enumeration. It is a
             * interface used to get elements of legacy
collections(Vector, Hashtable). So
             * Vectors can use both Enumeration and Iterator for
traversing over elements.
```

```
            */

           // creating Vector
           System.out.println(); // separation line
           Vector<String> v = new Vector<String>();
           v.addElement("red");
           v.addElement("blue");
           v.addElement("green");

           // traversing elements using Enumeration
           System.out.println("Vector elements: ");
           Enumeration e = v.elements();
           while (e.hasMoreElements())
                 System.out.println(e.nextElement());
           System.out.println(); // separation line
           /**
            * Stack class is the child class of Vector. It follows LIFO
(Last in first out)
            * mechanism.
            *
            * Stack methods: 1. Object push(Object item) - Pushes the
element into Stack
            * and returns the same element
            *
            * 2. Object pop() - Removes the element from Stack and
returns the element
            *
            * 3. Object peek() - Returns the top element from Stack.
            *
            * 4. Object empty() - Returns true when the Stack is empty.
            *
            * 5. int search(Object o) - Returns the offset value from
the stack if the
            * element is found, otherwise it returns -1.
            */

           Stack st = new Stack();
           st.push("Step by Step");
        st.push("in");
        st.push("Java");

        System.out.println("Top element from Stack: " + st.peek());

        while(!st.empty()){
            System.out.println("Element pops out: "+st.pop());
        }

        System.out.println("After all elements pop out is the Stack
empty? : "+st.empty());
      }
}
```

```
##--> 55Queues_PriorityQueue_AccessCard.java<-- ##

package com.udemy.app;

public class AccessCard implements Comparable<AccessCard> {

	private String name;
	private int idNumber;

	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}

	public int getIdNumber() {
		return idNumber;
	}

	public void setIdNumber(int idNumber) {
		this.idNumber = idNumber;
	}

	public AccessCard(String name, int idNumber) {
		this.name = name;
		this.idNumber = idNumber;
	}

	@Override
	public int compareTo(AccessCard otherCard) {
		// a version: assign priority to id number
		return Integer.compare(this.idNumber,
otherCard.getIdNumber());

		// b version: change order direction
		//return -Integer.compare(this.idNumber,
otherCard.getIdNumber());

		// c version: assign priority to name
		//return this.name.compareTo(otherCard.getName());
	}

	@Override
	public String toString() {
		return idNumber + "-" + name;
	}

}
```

```
##--> 55Queues_PriorityQueue_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Enumeration;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.PriorityQueue;
import java.util.Queue;
import java.util.Stack;
import java.util.Vector;

public class App {

      /**
       * Queue
       *
       * Queue: You use a queue when elements are added and removed in a
specific order. Queues are typically used
       * for sorting elements prior to processing them. Java Queue
interface orders the element in FIFO(First In
       * First Out) manner. In FIFO, first element is removed first and
last element is removed at last.
       *
       * PriorityQueue - ArrayDeque
       *
       * PriorityQueue
       * The PriorityQueue class provides the facility of using queue.
But it does not orders the elements in FIFO
       * manner. The elements of the priority queue are ordered according
to their natural ordering defined by the
       * Comparable interface.
       *
       * ArrayDeque
       * We will talk about it in the next lecture.
       *
       * Methods:
       *
       * add(object) add elements to the queue
       * offer(object) insert the specified element into this queue
       * remove() removes the head of this queue
       * element() retrieves, but does not remove, the head of this queue
       * peek() retrieves, but does not remove, the head of this queue,
or returns null if this queue is empty
       * poll() retrieves and removes the head of this queue, or returns
null if this queue is empty
       *
       */

      public static void main(String[] args) {
```

```
            // PriorityQueue example

            Queue<AccessCard> card = new PriorityQueue<>();
            card.add(new AccessCard("Alan", 2233));
            card.add(new AccessCard("Peter", 1100));
            card.add(new AccessCard("Joe", 7077));

            System.out.println("This element is on the head of this
queue: " + card.element());

            // card.remove();
            // card.offer(new AccessCard("Kevin", 3000));

            while (card.peek() != null) {
                  System.out.println(card.poll());
            }
      }
}
```

```
##--> 56Queues_ArrayDeque_App.java<-- ##

package com.udemy.app;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Deque;
import java.util.Enumeration;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;
import java.util.PriorityQueue;
import java.util.Queue;
import java.util.Stack;
import java.util.Vector;

public class App {

      /**
       * ArrayDeque
       *
       * ArrayDeque in Java provides a way to apply resizable-array in
addition to the implementation of the Deque
       * interface. It is also known as Array Double Ended Queue or Array
Deck. This is a special kind of array that
       * grows and allows users to add or remove an element from both the
sides of the queue. It's not ThreadSafe.
       *
       * add(object) add elements to the queue
       * offer(object) insert the specified element into this queue
       * remove() removes the head of this queue
       * element() retrieves, but does not remove, the head of this queue
       * peek() retrieves, but does not remove, the head of this queue,
or returns null if this queue is empty
       * poll() retrieves and removes the head of this queue, or returns
null if this queue is empty
       *
       * +
       *
       * addFirst() The method inserts particular element at the start of
the deque.
       * addLast() The method inserts particular element at the end of
the deque. It is similiar to add() method
       * removeFirst() : The method returns first element and also
removes it
       * removeLast() : The method returns last element and also removes
it
       **/

      public static void main(String[] args) {

            Deque<Integer> deque = new ArrayDeque<>();
```

```
            deque.add(10);
            deque.add(11);
            deque.add(12);
            deque.offerFirst(1);
            deque.offerLast(2);
            deque.offerFirst(3);
            deque.offerLast(4);

            //deque.removeLast();
            //deque.removeFirst();

            System.out.println("ArrayDeque contains: ");
            for(Integer i : deque){
            System.out.println(i);
            }

      }
}
```

```
##--> 57Sets_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Enumeration;
import java.util.HashSet;
import java.util.Iterator;
import java.util.LinkedHashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.PriorityQueue;
import java.util.Queue;
import java.util.Set;
import java.util.Stack;
import java.util.TreeSet;
import java.util.Vector;

public class App {

      /**
       * Sets
       *
       * You use a set when you don’t want to allow duplicate entries.
Booth of the sets are not synchronized. main
       * methods: add, remove, contains, etc.
       *
       * HashSet - LinkedHashSet - TreeSet
       *
       * A HashSet stores its elements in a hash table. This means that
uses the hashCode() method of the objects to
       * retrieve them more efficiently. HashSet the most common set!
       *
       * LinkedHashSet: If you need a HashSet that can be store items by
the insertion order.
       *
       * A TreeSet stores its elements in a sorted tree structure. The
main benefit is that the set is always in
       * sorted order. TreeSet is slower than HashSet.
       *
       **/

      public static void main(String[] args) {

            // HashSet
            Set<Integer> listHS = new HashSet<>();

            listHS.add(10);
            listHS.add(5);
            listHS.add(5);
            listHS.add(100);
            listHS.add(500);
```

```
            System.out.println(listHS.contains(500)); // contains method
            listHS.remove(500);
            System.out.println(listHS.contains(500)); // contains method

            System.out.println("-HashSet:");
            for (Integer i : listHS) {
                  System.out.println(i); // Elements in HashSet are not
ordered
            }

            // LinkedHashSet
            Set<Integer> listLHS = new LinkedHashSet<>();

            listLHS.add(10);
            listLHS.add(5);
            listLHS.add(5);
            listLHS.add(100);

            System.out.println("-LinkedHashSet:");
            for (Integer i : listLHS) {
                  System.out.println(i); // LinkedHashSet stores items by
the insertion order
            }

            // TreeSet
            Set<Integer> listTS = new TreeSet<>();

            listTS.add(10);
            listTS.add(5);
            listTS.add(5);
            listTS.add(100);

            System.out.println("-TreeSet:");
            for (Integer i : listTS) {
                  System.out.println(i); // TreeSet stores its elements in
a sorted order
            }

            // TreeSet from a HashSet
            /**
             * Because HashSet is faster, in general, if you want a
sorted set then it is better to add elements to
             * HashSet and then convert it into TreeSet rather than
creating a TreeSet and adding elements to it.
             */
            Set<Integer> listTS2 = new TreeSet<>(listHS);
            System.out.println("-TreeSet from HashSet:");
            for (Integer i : listTS2) {
                  System.out.println(i);
            }
      }
}
```

```
##--> 58Maps_App.java<-- ##

package com.udemy.app;

import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.TreeMap;

public class App {

      /**
       * Maps
       *
       * Map can store pairs of keys and values. Each key is linked to a
specific value. Once stored in a Map, you
       * can later look up the value using just the key.
       *
       * HashMap - Hashtable - LinkedHashMap - TreeMap
       *
       * HashMap doesn’t allow duplicate keys but allows duplicate
values. HashMap is implemented as a hash table,
       * and there is no ordering on keys or values. Hashtable is quite
similar to HashMap. The main difference is
       * that HashMap is not synchronized (not thread-safe), therefore
it’s faster and uses less memory than
       * Hashtable. Hashtable is slower, but it's synchronized (thread-
safe). + Hashtable doesn't allow duplicate
       * values.
       *
       * LinkedHashMap preserves the insertion order
       *
       * A TreeMap stores the keys in a sorted tree structure. The main
benefit is that the keys are always in
       * sorted order.
       *
       * Main methods:
       *
       * Object put(Object k, Object v) Puts an entry in the invoking
map, overwriting any previous value associated
       * with the key.
       * get(Object k) Returns the value associated with the key k.
       * int size() Returns the number of key/value pairs in the map.
       * clear() Removes all key/value pairs from the invoking map. etc.
       * entrySet() Returns a Set that contains the entries in the map.
       **/

      public static void main(String[] args) {

            // HashMap
            Map<Integer, String> mapHM = new HashMap<>();
            mapHM.put(155, "Joe");
            mapHM.put(90, "Tom");
            mapHM.put(30, "Adam");
```

```
            mapHM.put(101, "Adam");
            mapHM.put(10, "Kate");
            mapHM.put(10, "Judy"); // rewrite the VALUE of the KEY 10 to
Judy

            System.out.println("Value of the Key 10 is: " +
mapHM.get(10)); // Judy: the second VALUE of KEY10
            System.out.println("In the HashMap the elements number is: "
+ mapHM.size()); // 5 because KEY10's VALUE is

                                                              // overwritten

            System.out.println("- HashMap - no ordering");
            for (Map.Entry<Integer, String> entry : mapHM.entrySet()) {
                  System.out.println(entry.getKey() + " - " +
entry.getValue());
            }

            // LinkedHashMap
            Map<Integer, String> mapLHM = new LinkedHashMap<>();
            mapLHM.put(155, "Joe");
            mapLHM.put(90, "Tom");
            mapLHM.put(30, "Adam");
            mapLHM.put(101, "Adam");
            mapLHM.put(10, "Judy");

            System.out.println("- LinkedHashMap - insertion order");
            for (Map.Entry<Integer, String> entry : mapLHM.entrySet()) {
                  System.out.println(entry.getKey() + " - " +
entry.getValue());
            }

            // TreeMap
            Map<Integer, String> mapTM = new TreeMap<>();
            mapTM.put(155, "Joe");
            mapTM.put(90, "Tom");
            mapTM.put(30, "Adam");
            mapTM.put(101, "Adam");
            mapTM.put(10, "Judy");

            System.out.println("- TreeMap - sorted order");
            for (Map.Entry<Integer, String> entry : mapTM.entrySet()) {
                  System.out.println(entry.getKey() + " - " +
entry.getValue());
            }

            // TreeMap from HashMap
            Map<Integer, String> mapTM2 = new TreeMap<>(mapHM);
            System.out.println("- TreeMap from HashMap - sorted order");
            for (Map.Entry<Integer, String> entry : mapTM2.entrySet()) {
                  System.out.println(entry.getKey() + " - " +
entry.getValue());
            }
      }
```

}

```
##--> 60LocalInnerClass_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Local Inner Class
       *
       * Local Inner Class is a non-static class that is created inside a
class but inside a method.
       * Local Inner class is accessing the instance variable member of
the Outer class.
       *
       **/

      public static void main(String[] args) {

            OuterClass oClass = new OuterClass();
            oClass.outerMethod();
      }
}
```

```
##--> 60LocalInnerClass_OuterClass.java<-- ##

package com.udemy.app;

class OuterClass {
      private String msg = "Hello Local Inner Class!";// instance
variable

      void outerMethod() {
            class InnerClass {
                  void showMsg() {
                        System.out.println(msg);
                  }
            }
            InnerClass iClass = new InnerClass();
            iClass.showMsg();
      }
}
```

```
##--> 61MemberInnerClass_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Member Inner Class
       *
       * Member Inner Class created within class and outside method.
       * Member Inner class is accessing the instance variable member of
the Outer class.
       *
       **/

      public static void main(String[] args) {

            OuterClass oClass = new OuterClass();
            OuterClass.InnerClass iClass = oClass.new InnerClass();
            iClass.showMsg();
      }
}
```

```
##--> 61MemberInnerClass_OuterClass.java<-- ##

package com.udemy.app;

class OuterClass {
      private String msg = "Hello Member Inner Class!";// instance
variable

      class InnerClass {
            void showMsg() {
                  System.out.println(msg);
            }
      }
}
```

```
##--> 62StaticNestedClass_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Static Nested Class
       *
       * Static Nested Class created within class. And it's a static
class that is defined at the same level as
       * static variables. It's accessing the static variable member of
the Outer class, but it cannot access
       * non-static instance variables and methods of the outer class.
       * Nested class can be accessed without instantiating the outer
class.
       *
       **/

      public static void main(String[] args) {
            OuterClass.NestedClass nClass = new OuterClass.NestedClass();
            nClass.showMsg();
      }
}
```

```
##--> 62StaticNestedClass_OuterClass.java<-- ##

package com.udemy.app;

public class OuterClass {
	private static String msg = "Hello Static Nested Class!";// static
variable

	static class NestedClass {
		void showMsg() {
			System.out.println(msg);
		}
	}
}
```

```
##--> 63AnonymousInnerClass_AbstractClassForAnonymousClass.java<-- ##

package com.udemy.app;

abstract class AbstractClassForAnonymousClass {
      abstract void showMsg();
}
```

```
##--> 63AnonymousInnerClass_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Anonymous Inner Class
       *
       * Anonymous Inner Class created for implementing interface or
extending class.
       **/

      public static void main(String[] args) {
            AbstractClassForAnonymousClass aClass = new
AbstractClassForAnonymousClass() {
                  @Override
                  void showMsg() {
                        System.out.println("Hello Anonymous Inner Class
extending class!");
                  }
            };
            aClass.showMsg();

            /**
             * It's an example how the code looks like without Anonymous
Inner Class. In the background this happens when
             * Anonymous Inner Class is working with the difference that
Java doesn't give name to this class (that's why
             * it's anonymous)
             */
            WithoutAnonimous wa = new WithoutAnonimous();
            wa.showMsg();

            /**
             * Anonymous Inner Class implementing Interface
             */

            InterfaceForAnonymousClass aInterface = new
InterfaceForAnonymousClass() {
                  public void showMsg() {
                        System.out.println("Hello Anonymous Inner Class
implementing Interface!");
                  }
            };
            aInterface.showMsg();
      }
}
```

```
##--> 63AnonymousInnerClass_InterfaceForAnonymousClass.java<-- ##

package com.udemy.app;

public interface InterfaceForAnonymousClass{
      void showMsg();
}
```

```
##--> 63AnonymousInnerClass_WithoutAnonimous.java<-- ##

package com.udemy.app;

public class WithoutAnonimous extends AbstractClassForAnonymousClass {

      WithoutAnonimous(){}

            @Override
            void showMsg() {
                  System.out.println("Without Anonymous Inner Class
extending class!");
            }

}
```

```
##--> 64Exceptions_App.java<-- ##

package com.udemy.app;

import java.util.Scanner;

public class App {

      /**
       * Exceptions
       **/

      public static void main(String[] args) {

            /**
             * Catching Multiple Exception
             */

            Scanner scn = new Scanner(System.in);
        try
        {
            System.out.println("Give me a number: ");
             int n = Integer.parseInt(scn.nextLine());
             System.out.println(20/n);
        }
        catch (ArithmeticException ex)
        {
             System.out.println("Arithmetic " + ex);
        }
        catch (NumberFormatException ex)
        {
             System.out.println("Number Format Exception " + ex);
        }
        scn.close();
        System.out.println("more code...");
      }
}
```

```
##--> 65ExceptionsA_App.java<-- ##

package com.udemy.app;

import java.util.Scanner;

public class App {

	/**
	 * Exceptions
	 **/

	public static void main(String[] args) {

		/**
		 * Catching Multiple Exception
		 */

		Scanner scn = new Scanner(System.in);
		try {
			System.out.println("Give me a number: ");
			int n = Integer.parseInt(scn.nextLine());
			System.out.println(20 / n);
		} catch (ArithmeticException ex) {
			System.out.println("Arithmetic " + ex);
		} catch (NumberFormatException ex) {
			System.out.println("Number Format Exception " + ex);
		}
		scn.close();
		System.out.println("more code...");
	}
}
```

```
##--> 65ExceptionsB_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Exceptions
       **/

      public static void main(String[] args) {

            /**
             * Nested try block
             */

            // outer try-catch block
            try {
                  int exampleArray[] = { 1, 2, 3, 4, 5 };
                  System.out.println(exampleArray[9]);

                  // second, inner try-block inside another try block
                  try {
                        int x = exampleArray[3] / 0; // division
                        System.out.println("Result of division: " + x);
                  } catch (ArithmeticException e2) {
                        System.out.println("ArithmeticException - Division
by zero is not possible");
                  }

            } catch (ArrayIndexOutOfBoundsException e1) {
                  System.out.println("ArrayIndexOutOfBoundsException -
Element at such index does not exists");
            }
      }
}
```

```
##--> 65ExceptionsC_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Exceptions
       **/

      public static void main(String[] args) {

            /**
             * ExceptionHandling with MethodOverriding
             */

            ExceptionParent p = new ExceptionChild();
        p.method();

    }

}
```

```
##--> 65ExceptionsC_ExceptionChild.java<-- ##

package com.udemy.app;

import java.io.IOException;

public class ExceptionChild extends ExceptionParent {

	// void method() throws IOException{ // 1/a) example
	// void method() throws ArithmeticException{ // 1/b) example
	// void method() throws IOException{ // 2/a) example
	// void method() throws ArithmeticException{ // 2/b) example
	@Override
	void method() {
		System.out.println("Child Class");
	}
}
```

```
##--> 65ExceptionsC_ExceptionParent.java<-- ##

package com.udemy.app;

public class ExceptionParent {

      // void method(){ // 1.a) example
      // void method(){ // 1.b) example
      // void method() throws ArithmeticException{ // 2/a) example
      // void method() throws ArithmeticException{ // 2/b) example
      void method() {
            System.out.println("Parent Class");
      }
}
```

```
##--> 68InheritanceTypes_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Inheritance
       *
       ****
       * Single-level Inheritance
       *
       * Base Class - Derived Class
       *
       ****
       * Multilevel Inheritance
       *
       * Base Class - Intermediatory Class - Derived Class
       * A class can inherit from a derived class too, thereby making
this derived class (Intermediatory Class)
       * the base class for the new class.
       *
       ****
       * Hierarchical Inheritance
       * One class serves as a base (super) class for more than one sub
class
       *
       *                 - Derived Class A
       * Base Class - Derived Class B
       *                 - Derived Class C
       *
       ****
       * Multiple Inheritance
       * In Multiple inheritance ,one class can have more than one
superclass and inherit features from all parent
       * classes. Please note that Java does not support multiple
inheritance with classes. In java, we can achieve
       * multiple inheritance only through Interfaces.
       *
       * interface A -
       * interface B - Derived Class
       *
       ****
       * Hybrid Inheritance
       * It is a mix of two or more of the above types of inheritance.
       * For example:
       *
       * Interface A -
       * Base class  - Derived Class
       * Interface B -
       *
       **/

      public static void main(String[] args) {
```

```
            /**
             * Single-level Inheritance example
             *
             * Parent - Child
             */
            System.out.println("- Single-level Inheritance -");

            Child child = new Child();
            child.greatingsFromParent();
            child.greatingsFromChild();

            /**
             * Multilevel Inheritance example
             *
             * Parent - Child - Grandchild
             */

            System.out.println("- Multilevel Inheritance -");

            Grandchild grandChild = new Grandchild();
            grandChild.greatingsFromParent();
            grandChild.greatingsFromChild();
            grandChild.greatingsFromGrandchild();

             /** Hierarchical Inheritance example
             *
             * Parent - Child
             *            - Child2
             */

            System.out.println("- Hierarchical Inheritance -");
            Child c = new Child();
            Child2 c2 = new Child2();

            c.greatingsFromParent();
            c.greatingsFromChild();
            c2.greatingsFromParent();
            c2.greatingsFromChild2();

            /**
             * Multiple Inheritance example
             *
             * Interface1 -
             * Interface2 - (Interface3) - MultipleInheritanceClass
             */

            System.out.println("- Multiple Inheritance -");
            MultipleInheritanceClass mClass = new
MultipleInheritanceClass();
            mClass.method1();
            mClass.method2();

            /**
```

```
             * Hybrid Inheritance example
             *
             * Interface1 -
             * Interface2 - (Interface3) - HybridInheritanceClass
             * Parent     -
             */

            System.out.println("- Hybrid Inheritance -");
            HybridInheritanceClass hClass = new HybridInheritanceClass();
            hClass.method1();
            hClass.method2();
            hClass.greatingsFromParent();



        }
    }
```

```
##--> 68InheritanceTypes_Child.java<-- ##

package com.udemy.app;

public class Child extends Parent{

      protected void greatingsFromChild() {
            System.out.println("hello from Child");
      }
}
```

```
##--> 68InheritanceTypes_Child2.java<-- ##

package com.udemy.app;

public class Child2 extends Parent{

      protected void greatingsFromChild2() {
            System.out.println("hello from Child2");
      }
}
```

```
##--> 68InheritanceTypes_Grandchild.java<-- ##

package com.udemy.app;

public class Grandchild extends Child{

      protected void greatingsFromGrandchild() {
            System.out.println("hello from Grandchild");
      }

}
```

```
##--> 68InheritanceTypes_HybridInheritanceClass.java<-- ##

package com.udemy.app;

public class HybridInheritanceClass extends Parent implements Interface3{

	@Override
	public void method1() {
		System.out.println("method of Interface 1 overriding");
	}

	@Override
	public void method2() {
		System.out.println("method of Interface 2 overriding");
	}

}
```

```
##--> 68InheritanceTypes_Interface1.java<-- ##

package com.udemy.app;

public interface Interface1{

      public void method1();

}
```

```
##--> 68InheritanceTypes_Interface2.java<-- ##

package com.udemy.app;

public interface Interface2{

      public void method2();

}
```

```
##--> 68InheritanceTypes_Interface3.java<-- ##

package com.udemy.app;

public interface Interface3 extends Interface1, Interface2{

      //It's extends Interface1, Interface2 so it's "contains" the
methods of their

}
```

```
##--> 68InheritanceTypes_MultipleInheritanceClass.java<-- ##

package com.udemy.app;

public class MultipleInheritanceClass implements Interface3{

	@Override
	public void method1() {
		System.out.println("method of Interface 1 overriding");
	}

	@Override
	public void method2() {
		System.out.println("method of Interface 2 overriding");
	}

}
```

```
##--> 68InheritanceTypes_Parent.java<-- ##

package com.udemy.app;

public class Parent {

      protected void greatingsFromParent() {
            System.out.println("hello from Parent");
      }
}
```

```
##--> 69Polymorphism_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Polymorphism with Multilevel Inheritance &
       * Runtime Polymorphism with Data Members
       *
       **/

      public static void main(String[] args) {

            /**
             * Polymorphism with Multilevel Inheritance
             */

            Parent p1 = new Parent();
            Parent p2 = new Child();
            Parent p3 = new GrandChild();

            p1.travel();
            p2.travel();
            p3.travel();

            /**
             * Runtime Polymorphism with Data Members
             *
             * In Java, we can override methods only, not the
variables(data members), so runtime polymorphism cannot be
             * achieved by data members.
             */
            System.out.println("Age: " + p1.age);
            System.out.println("Age: " + p2.age);
            System.out.println("Age: " + p3.age);

            /**
             * Since variables are not overridden, the p1.age, p2.age and
p3.age statements will always refer to data
             * member of super class.
             */

            // only in this case we will access the GrandChild's age
            GrandChild g = new GrandChild();
            System.out.println("Age: " + g.age);
      }
}
```

```
##--> 69Polymorphism_Child.java<-- ##

package com.udemy.app;

public class Child extends Parent {

	protected int age = 30;

	@Override
	protected void travel() {
		System.out.println("Child is traveling with car");
	}

}
```

```
##--> 69Polymorphism_GrandChild.java<-- ##

package com.udemy.app;

public class GrandChild extends Child{

      protected int age = 10;

      @Override
      protected void travel() {
            System.out.println("GrandChild is traveling with bike");
      }

}
```

```
##--> 69Polymorphism_Parent.java<-- ##

package com.udemy.app;

public class Parent {

	protected int age = 50;

	protected void travel() {
		System.out.println("Parent is traveling with airplane");
	}

}
```

```
##--> 70AdvancedEncapsulation_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;

public class App {

      /**
       * Advanced Encapsulation
       * Encapsulation:
       * 1. fields set to private: restricted access
       * 2. use setters and getters
       * + for advanced: check validity of the setting data
       **/

      public static void main(String[] args) {

            /**
             * Advanced Encapsulation example
             */

            Person p = new Person();
            p.setName("Kevin");
            p.setAge(20);
            //p.setAge(-20);
            System.out.println("The person's name is: " + p.getName() + "
and the person's age is: " + p.getAge());

      }
}
```

```
##--> 70AdvancedEncapsulation_Person.java<-- ##

package com.udemy.app;

public class Person {

	private String name;
	private int age;

	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}

	public int getAge() {
		return age;
	}

	/**
	 * 1. Simple setter
	 */

	public void setAge(int age) {
		this.age = age;
	}

	/**
	 * 2. Advanced Encapsulation example
	 * set the "age" variable to the new value, otherwise you ignore
	 */

//	public void setAge(int age) {
//		if (age >= 0) {
//			this.age = age;
//		}
//	}

	/**
	 * 3. Advanced Encapsulation example
	 * improved version - set the "age" variable to the new value,
otherwise throw exception
	 */

//	public void setAge(int age) {
//		if (age < 0) {
//			throw new IllegalArgumentException("Age cannot be a
negative number");
//		}
//		this.age = age;
//	}
```

}

```
##--> 71EncapsulationWithMutableClasses_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;

public class App {

      /**
       * Encapsulation with Mutable Classes
       * We must be careful when encapsulating mutable objects.
       **/

      public static void main(String[] args) {

            /**
             * Encapsulation with Mutable Classes example
             */

            ArrayList<String> brands = new ArrayList<String>();
            brands.add("Ford");
            brands.add("BMW");
            brands.add("Honda");
            Cars cars = new Cars(brands); // through the constructor we
initialize Cars class carNames Arraylist

            for (String carNames : cars.getCarNames()) { // print out the
elements
                  System.out.print(carNames + " ");
            }

            ArrayList<String> localArrayList = cars.getCarNames(); //
Storing the return value
            localArrayList.add("Toyota"); // Changing the object returned

            System.out.println(); // new line
            for (String carNames : cars.getCarNames()) { // print out
again the elements from encapsulated class
                  System.out.print(carNames + " ");
            }

      }
}
```

```
##--> 71EncapsulationWithMutableClasses_Cars.java<-- ##

package com.udemy.app;

import java.util.ArrayList;

public class Cars {

	private ArrayList<String> carNames;

	public Cars(ArrayList<String> carNames) {
		this.carNames = carNames;
	}

	public ArrayList<String> getCarNames() {
		return carNames;
	}

	//solution
//	public ArrayList<String> getCarNames() {
//		return new ArrayList<String>(carNames);
//	}

}
```

```
##--> 72JavaBeans_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * JavaBeans
       * JavaBeans are classes that encapsulate many objects into a
single object (the bean).
       * JavaBeans should follow following conventions:
       * 1. Must implement Serializable
       * 2. It should have a public zero-argument constructor
       * 3. All properties must be private and allow access to properties
using getter and setter methods
       **/

      public static void main(String[] args) {

            /**
             * JavaBeans example
             */

            //to access JavaBean class
            JavaBeanExample bean = new JavaBeanExample(); // object is
created
            bean.setName("Bean Example"); // setting value to the object
            bean.setId(199);
        System.out.println(bean.getName() + ", " + bean.getId());

      }
}
```

```
##--> 72JavaBeans_JavaBeanExample.java<-- ##

package com.udemy.app;

public class JavaBeanExample implements java.io.Serializable {

	private int id;
	private String name;

	public JavaBeanExample() {
	}

	public int getId() {
		return id;
	}

	public void setId(int id) {
		this.id = id;
	}

	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}

}
```

```
##--> 73AbstractionMoreAboutInterface_AnotherInterface.java<-- ##

package com.udemy.app;

public interface AnotherInterface {

	//default method
	public default void showDefault() {
		System.out.println("AnotherInterface's default method");
	};
}
```

```
##--> 73AbstractionMoreAboutInterface_App.java<-- ##

package com.udemy.app;

//public class App implements ModernInterface, AnotherInterface
public class App implements ModernInterface {

      /**
       * Modern Interface
       * Originally an interface could only have abstract methods, but
with JDK 8 changes have come. From JDK 8 the
       * interface body can contain abstract methods, default methods,
and static methods too.
       **/

      public static void main(String[] args) {

            App a = new App();

            // abstract method calling
            a.showAbstract();

            // static method calling
            ModernInterface.showStatic();

            // default method calling
            a.showDefault();

      }

      @Override
      public void showAbstract() {
            System.out.println("ModernInterface's abstract method
overrided");
      }

      /**
       * Multiple Defaults
       *
       * With default functions in interfaces, there is a possibility
that a class is implementing two interfaces
       * with same default methods.
       *
       * 1. We can override
       * 2. We can call the default method of the specified interface
using super
       */

//    public void showDefault() {
//          System.out.println("Overrided default method"); // override
//          //ModernInterface.super.showDefault(); // method from
ModernInterface
//          //AnotherInterface.super.showDefault(); // method from
AnotherInterface
```

```
//     }

}
```

```
##--> 73AbstractionMoreAboutInterface_ModernInterface.java<-- ##

package com.udemy.app;

/**
 * Modern Interface
 */

public interface ModernInterface {

      //abstract method
      public void showAbstract();

      //static method
      public static void showStatic() {
            System.out.println("ModernInterface's static method");
      }

      //default method
      public default void showDefault() {
            System.out.println("ModernInterface's default method");
      };

}
```

##--> 74AbstractionNestedInterface_App.java<-- ##

```
package com.udemy.app;

public class App implements OuterInterface.NestedInterface,
OuterClass.NestedInterfaceInClass {

	/**
	 * Abstraction - Nested Interface
	 *
	 * An interface which is declared inside another interface or class
is called nested (or inner) interface. The
	 * nested interfaces are used to group related interfaces so that
they can be easy to maintain. The nested
	 * interface must be referred by the outer interface or class. It
can't be accessed directly.
	 **/

	public static void main(String[] args) {

		/**
		 * Nested Interface in Interface
		 */
		OuterInterface.NestedInterface msg = new App(); // upcasting
		msg.messageIn();

		/**
		 * Nested Interface in Class
		 */
		OuterClass.NestedInterfaceInClass msg2 = new App();
		msg2.show();

	}

	/**
	 * Nested Interface in Interface
	 */
	@Override
	public void messageIn() {
		System.out.println("NestedInterface's method overriding");
	}

	/**
	 * Nested Interface in Class
	 */
	@Override
	public void show() {
		System.out.println("NestedInterface from Class method
overriding");
	}
}
```

```
##--> 74AbstractionNestedInterface_OuterClass.java<-- ##

package com.udemy.app;

/**
 * Nested Interface in Class
 *
 * Nested interfaces are static by default. You don’t have to mark them
static explicitly as it would be
 * redundant.
 * Nested interfaces declared inside class can take any access modifier,
however nested interface declared
 * inside interface is public implicitly.
 */

public class OuterClass {

      public interface NestedInterfaceInClass {

            public void show();
      }
}
```

```
##--> 74AbstractionNestedInterface_OuterInterface.java<-- ##

package com.udemy.app;

/**
 * Nested Interface in Interface
 *
 * Nested interfaces are static by default. You don’t have to mark them
static explicitly as it would be
 * redundant.
 * Nested interfaces declared inside class can take any access modifier,
however nested interface declared
 * inside interface is public implicitly.
 */

public interface OuterInterface {

      public void messageOut();

      public interface NestedInterface {

            public void messageIn();
      }
}
```

```
##--> 76Serialization_Car.java<-- ##

package com.udemy.app;

import java.io.Serializable;

public class Car implements Serializable {

	private static final long serialVersionUID = 3658213896628538076L;

	String carBrand;
	int motorNumber;
	String color;
	// transient String color; // transient = it will not be serialized

	public Car(String carBrand, int motorNumber, String color) {
		this.carBrand = carBrand;
		this.motorNumber = motorNumber;
		this.color = color;
	}
}
```

```
##--> 76Serialization_Ford.java<-- ##

package com.udemy.app;

public class Ford extends Car {

      /**
       * Serialization with Inheritance
       * If a class implements serializable then all its sub classes will
also be serializable.
       */

      String fordType;

      public Ford(String carBrand, int motorNumber, String color, String
fordType) {
            super(carBrand, motorNumber, color);
            this.fordType = fordType;
      }

}
```

```
##--> 76Serialization_Serialization.java<-- ##

package com.udemy.app;

import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;

public class Serialization {

      /**
       * Serialization
       *
       **/

      public static void main(String args[]) throws Exception {

            /**
             * Serialization
             */

            Car car = new Car("Ford", 2110022, "green");

            try {
                  // Saving the object in a file
                  FileOutputStream fileOut = new
FileOutputStream("f.txt");
                  ObjectOutputStream out = new
ObjectOutputStream(fileOut);

                  // Method for serialization of object
                  out.writeObject(car);

                  out.close();
                  fileOut.close();

                  System.out.println("Object has been serialized");
            } catch (IOException ex) {
                  System.out.println("IOException is caught");
            }

            /**
             * Deserialization
             */

            try {
                  // Reading the object from a file
                  FileInputStream file = new FileInputStream("f.txt");
                  ObjectInputStream in = new ObjectInputStream(file);

                  // Method for deserialization of object
                  Car car2 = (Car) in.readObject();
```

```
                in.close();
                file.close();

                System.out.println("Object has been deserialized ");
                System.out.println(car2.carBrand + " " +
car2.motorNumber + " " + car2.color);
            } catch (IOException ex) {
                System.out.println("IOException is caught");
            }

      }

}
```

```
##--> 78LambdaExpression_App.java<-- ##

package com.udemy.app;

public class App {

      /**
       * Lambda expression
       *
       **/

      public static void main(String[] args) {

            /**
             * Lambda expression example
             *
             **/

            // without lambda, Sum implementation using anonymous
implementation of interface
            Sum s = new Sum() {
                  @Override
                  public int adding(int i, int i2) {
                        return i + i2;
                  }
            };
            System.out.println(s.adding(4, 2));

            // with lambda
            Sum s2 = (int i, int i2) -> {return i+i2;};

            //simplified lambda expression
            //Sum s2 = (i, i2) -> i+i2;

            System.out.println(s2.adding(5, 3));

            /**
             * Lambda expression without parameter or with one parameter
example
             *
             **/

            //without parameter interface method case
            Greatings g = () -> System.out.println("Hello ");
            g.sayHello();


            //with one parameter interface method case
//          Greatings g = str -> System.out.println("Hello " + str);
//          g.sayHello("Java");


      }
```

}

```
##--> 78LambdaExpression_Greatings.java<-- ##

package com.udemy.app;

public interface Greatings {
      //without parameter case
      public void sayHello();

      //with one parameter case
      //public void sayHello(String text);
}
```

```
##--> 78LambdaExpression_Sum.java<-- ##

package com.udemy.app;

public interface Sum {
      public int adding(int i, int i2);
}
```

```
##--> 79FunctionalInterface_App.java<-- ##

package com.udemy.app;

import java.util.function.Consumer;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.function.Supplier;
import java.util.function.UnaryOperator;

public class App {

      /**
       * Functional interfaces
       **/

      public static void main(String[] args) {

            // simplified lambda expression
            Sum sum = (i, i2) -> i + i2;
            System.out.println(sum.adding(5, 3));

            /**
             * Predefined functional interfaces - Consumer<T>
             */

            // Consumer interface represents an operation that accepts a
single input argument and returns no result.
            Consumer<String> cons = (st) ->
System.out.println(st.toLowerCase());
            cons.accept("Java programing: Step by Step from A to Z");
            // accept: performs operation on the given argument.

            /**
             * Predefined functional interfaces - Predicate<T>
             */

            // Predicate interface represents a predicate (boolean-valued
function) of one argument.
            Predicate<Integer> pr = a -> (a > 10);
            System.out.println(pr.test(11));
            // test: evaluates this predicate on the given argument.

            /**
             * Predefined functional interfaces - UnaryOperator<T>
             */

            // It represents an operation on a single operand that
returns a result of the same type as its operand
            UnaryOperator<String> str = (x) -> x.toUpperCase();
            System.out.println(str.apply("hello java"));

            /**
             * It represents a supplier of results. - Supplier<T>
```

```
            */

            // It represents a supplier of results.
            // The supplier can be used in all contexts where there is no
input but an output is expected.
            Supplier<Double> randomValue = () -> Math.random();
            System.out.println("Supplier random value: " +
randomValue.get());

            /**
             * Predefined functional interfaces - Function<T,R>
             */

            // It represents a function that accepts one argument and
returns a result.
            Function<String, String> functionExample = s -> {
                  return message(s);
            };
            System.out.println(functionExample.apply("World!"));

      }

      static String message(String s) {
            return "Hello " + s;
      };

}
```

```
##--> 79FunctionalInterface_Sum.java<-- ##

package com.udemy.app;

/**
 * Any interface with a SAM (Single Abstract Method) is a functional
interface, and its implementation may be
 * treated as lambda expressions.
 */

@FunctionalInterface
public interface Sum {
      public int adding(int i, int i2);

      // public int secondAbstractMethod(); //gives error, because of the
SAM rule

      static void staticMethodExample() { // it's permitted, because of
the SAM rule
            // code here
      };

      default void defMethodExample() { // it's permitted, because of the
SAM rule
            // code here
      }

      /**
       * Notice! The "SAM" speaks only about Abstract Method, so in a
@FunctionalInterface there can be only one of
       * Abstract Method, BUT as we have learned about Interfaces we can
have default and static methods too. So in
       * an @FunctionalInterface default and static methods are
permitted.
       */
}
```

```
##--> 81MethodReference_App.java<-- ##

package com.udemy.app;

import java.util.Arrays;

interface FunctionalInterface {
      void show();
}

interface FunctionalInterface2 {
      MessageClass showMessage(String msg);
}

interface Sum {
      public int adding(int i, int i2);
}

public class App {

      /**
       * Method References
       *
       * There are four kinds of method references:
       * 1. Reference to a Static Method
       * 2. Reference to an Instance Method of a Particular Object
       * 3. Reference to an Instance Method of an Arbitrary Object of a
Particular Type
       * 4. Reference to a Constructor
       *
       **/

      public static void main(String[] args) {

            /**
             * 1. Reference to a Static Method
             */
            // Referring static method
            FunctionalInterface funcInt = App::staticGreeting;
            // Calling interface method
            funcInt.show();

            /**
             * 2. Reference to an Instance Method of a Particular Object
             */
            // A.) refer methods by class object
            App a = new App();
            FunctionalInterface f = a::instanceGreeting;
            f.show();
            // B.) refer methods by anonymous object
            FunctionalInterface f2 = new App()::instanceGreeting;
            f2.show();

            /**
```

```
            * 3. Reference to an Instance Method of an Arbitrary Object
of a Particular Type
            */

           String[] stringArray = { "Kevin", "Alan", "Joe" };
           Arrays.sort(stringArray, String::compareToIgnoreCase);
           System.out.println("Array's sorted elements: " +
Arrays.toString(stringArray));

           /**
            * 4. Reference to a Constructor
            */
           FunctionalInterface2 f3 = MessageClass::new;
           f3.showMessage("Hello, this is Reference to a Constructor
case.");
      }

      public static void staticGreeting() {
           System.out.println("Hello, this is static method.");
      }

      public void instanceGreeting() {
           System.out.println("Hello, this is instance, non-static
method.");
      }
}
```

```
##--> 81MethodReference_MessageClass.java<-- ##

package com.udemy.app;

public class MessageClass {

      MessageClass(String msg){
         System.out.print(msg);
    }
}
```

```
##--> 82MethodReferenceVsLambda_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;

public class App {

      /**
       * Method Reference vs Lambda
       *
       **/

      public static void main(String[] args) {

            /**
             * 1.) method reference better in this case because of its
simplicity
             */

            // lambda
            Sum s = (i, i2) -> add(i, i2);
            System.out.println(s.adding(8, 9));

            // method reference
            Sum s2 = App::add;
            System.out.println(s2.adding(8, 9));

            /**
             * 2.) lambda expression works only in this case
             */

            // lambda
            Sum s3 = (i, i2) -> add(i, increaseValue(i2));
            System.out.println(s3.adding(8, 9));

            // method reference
            /**
             * This simple case can’t be expressed with a method
reference because method reference is just a shorthand
             * syntax for a SIMPLE lambda expression that executes just
one method without doing anything else. If you
             * need adaptations or additional operations, there is no
special syntax for method references to support
             * that.
             */

      }

      public static int add(int i, int i2) {
            return i + i2;
      }

      public static int increaseValue(int x) {
```

```
            x = x + 33;
            return x;
        }

    }
```

```
##--> 82MethodReferenceVsLambda_Sum.java<-- ##

package com.udemy.app;

@FunctionalInterface
public interface Sum {
      public int adding(int i, int i2);
}
```

```
##--> 83forEachWithLambdaAndMethodReference_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

public class App {

      /**
       * forEach with Lambda and Method reference
       *
       * Introduced in Java 8, the forEach loop provides programmers with
a new, simplified way for
       * iterating over a collection.
       **/

      public static void main(String[] args) {

            /**
             * List
             */

            List<String> names = new ArrayList<>();
            names.add("Kevin");
            names.add("Alan");
            names.add("Joe");

            // enhanced for loop version of iterating
            System.out.println("for-loop version of iterating:");
            for (String s : names) {
                  System.out.println(s);
            }

            // with Lambda expression
            System.out.println("using forEach with Lambda:");
            names.forEach(s -> System.out.println(s));

            // with Method Reference
            System.out.println("using forEach with Method Reference:");
            names.forEach(System.out::println);

            /**
             * Set
             */

            // works on HashSet
            System.out.println("using forEach on HashSet:");
```

```
            Set<String> uniqueNames = new
HashSet<>(Arrays.asList("Larry", "Steve", "James"));
            uniqueNames.forEach(System.out::println);

            /**
             * Map
             */

            // enhanced for loop version of iterating
            Map<String, Integer> items = new HashMap<>();
            items.put("A", 10);
            items.put("B", 20);
            items.put("C", 30);

            System.out.println("using forEach on Map:");
            for (Map.Entry<String, Integer> entry : items.entrySet()) {
                  System.out.println("Item : " + entry.getKey() + " Count
: " + entry.getValue());
            }

            // lambda
            System.out.println("using forEach on Map with Lambda:");
            items.forEach((k, v) -> System.out.println("Item : " + k + "
Count : " + v));

            // method reference
            // items.forEach(System.out::println);
            // Error: The type PrintStream does not define
println(String, Integer) that is applicable here

            /**
             * we can use method reference in cases where Lambda body
only contains a call to another method. But since
             * there are two implicit variables ( key and value) and we
cannot use the below code to call println() method
             * of System.out class.
             */
      }

}
```

```
##--> 85OptionalOverviewExamples_OptionalClass.java<-- ##

package com.udemy.app;

import java.util.Optional;

public class OptionalClass {

      /**
       * Optional
       **/

      public static void main(String[] args) {

            /**
             * empty()
             * of()
             * ofNullable()
             */
            System.out.println("- Optional : empty(), of(), ofNullable()
-");

            Person person = new Person("Kevin Smith");

            Optional<Person> personE = Optional.empty();
            System.out.println(personE);

            Optional<Person> personOptional = Optional.of(person);
            // Optional<Person> personOptional = Optional.of(null);
//Throw NullPointerException

            Optional<Person> personOptional2 = Optional.ofNullable(null);

            /**
             * ifPresent()
             * get()
             * isPresent()
             */
            System.out.println("\n- Optional : ifPresent(), get(),
isPresent() -");

            //Optional<Person> perOp = Optional.ofNullable(null);
            Optional<Person> perOp = Optional.ofNullable(person);

            // ifPresent()
            perOp.ifPresent(p -> System.out.println(p.getName()));

            // get()
            System.out.println("Value found : " + perOp.get().getName());
// if Optional empty: NoSuchElementException

            // isPresent()
            if (perOp.isPresent()) {
                  // value is present inside Optional
```

```
                System.out.println("Value found : " +
perOp.get().getName());
            } else {
                // value is absent
                System.out.println("Optional is empty");
            }

            /**
             * orElse()
             * orElseGet()
             * or()
             */
            System.out.println("\n- Optional : orElse(), orElseGet(),
or() -");

            Optional<Person> personOriginal = Optional.ofNullable(new
Person("Peter Parker"));
            // Optional<Person> personOriginal =
Optional.ofNullable(null);

            // example A - orElse
            Person personA = personOriginal.orElse(new Person("Joe
Black")); // orElse : if personOptional empty
            System.out.println(personA.getName());

            // example B - orElseGet
            Person personB = personOriginal.orElseGet(() ->
giveMeAnotherPerson()); // orElseGet : if personOptional empty
            System.out.println(personB.getName());

            // example C - or
            Optional<Person> personC = Optional.ofNullable(new
Person("Clark Kent"));
            Optional<Person> personC2 = personOriginal.or(() -> personC);
// or : if personOptional empty
            System.out.println(personC2.get().getName());

            /**
             * filter()
             */
            System.out.println("\n- Optional : filter() -");

            personOptional.filter(personOpt ->
personOpt.getName().equals("Kevin Smith"))
                        .ifPresent(p -> System.out.println("Filtering with
the value 'Kevin Smith' found"));

      }

      static Person giveMeAnotherPerson() {
            Person per = new Person("Amanda Fox");
            return per;
      }
```

}

```
##--> 85OptionalOverviewExamples_Person.java<-- ##

package com.udemy.app;

public class Person {
	String name;

	public Person(String name) {
		this.name = name;
	}

	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}

}
```

```
##--> 86A_NonOptionalExample_NonOptionalExample.java<-- ##

package com.udemy.app;

public class NonOptionalExample {

      /**
       * NonOptional example as comparison
       **/

      public static void main(String[] args) {

            Stamp stamp = new Stamp("Barbados");
            //Stamp stamp = null;
            Passport passport = new Passport(stamp);
            //Passport passport = null;
            Person person = new Person("Kevin Smith", passport);
            //Person person = null;

            /**
             * If person, passport or stamp is null, than
NullPointerExecption
             */

      //System.out.println(person.getPassport().getStamp().getCountryOfSt
amp());

            String country = "Unknown";

            if (person != null) {
                  Passport pass = person.getPassport();
                  if (pass != null) {
                        Stamp st = passport.getStamp();
                        if (st != null) {
                              country =
person.getPassport().getStamp().getCountryOfStamp();
                        }
                  }
            }
            System.out.println(country);

      }

}
```

```
##--> 86A_NonOptionalExample_Passport.java<-- ##

package com.udemy.app;

public class Passport {
	private Stamp stamp;

	public Passport(Stamp stamp) {
		this.stamp = stamp;
	}

	public Stamp getStamp() {
		return stamp;
	}

	public void setStamp(Stamp stamp) {
		this.stamp = stamp;
	}
}
```

```
##--> 86A_NonOptionalExample_Person.java<-- ##

package com.udemy.app;

public class Person {
	private String name;
	private Passport passport;

	public Person(String name, Passport passport) {
		this.name = name;
		this.passport = passport;
	}

	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}

	public Passport getPassport() {
		return passport;
	}

	public void setPassport(Passport passport) {
		this.passport = passport;
	}


}
```

```
##--> 86A_NonOptionalExample_Stamp.java<-- ##

package com.udemy.app;

public class Stamp {
	private String countryOfStamp;

	public String getCountryOfStamp() {
		return countryOfStamp;
	}

	public void setCountryOfStamp(String countryOfStamp) {
		this.countryOfStamp = countryOfStamp;
	}

	public Stamp(String dateOfStamp) {
		this.countryOfStamp = dateOfStamp;
	}
}
```

```
##--> 86B_OptionalExample_OptionalExample.java<-- ##

package com.udemy.app;

import java.util.Optional;

public class OptionalExample {

      /**
       * Optional
       **/

      public static void main(String[] args) {

            Optional<Stamp> stamp = Optional.ofNullable(new
Stamp("Barbados"));
            //Optional<Stamp> stamp = Optional.ofNullable(null);
            Optional<Passport> passport = Optional.ofNullable(new
Passport(stamp));
            //Optional<Passport> passport = Optional.ofNullable(null);
            Optional<Person> person = Optional.ofNullable(new
Person("Kevin", passport));
            //Optional<Passport> person = Optional.ofNullable(null);



            /**
             * with maps:
             * Compile time error: The type Passport does not define
getStamp(Optional<Passport>) that is applicable here
             */
//          String ld = person.map(Person::getPassport)
//                      .map(Passport::getStamp)
//                      .map(Stamp::getCountryOfStamp)
//                      .orElse("Unknown");

            /**
             * combination of map and flatMap works
             */
            String result = person.flatMap(Person::getPassport)
                        .flatMap(Passport::getStamp)
                        .map(Stamp::getCountryOfStamp)
                        .orElse("Unknown");

            System.out.println(result);

      }

}
```

```
##--> 86B_OptionalExample_Passport.java<-- ##

package com.udemy.app;

import java.util.Optional;

public class Passport {
	private Optional <Stamp> stamp;

	public Passport(Optional<Stamp> stamp) {
		this.stamp = stamp;
	}

	public Optional<Stamp> getStamp() {
		return stamp;
	}

	public void setStamp(Optional<Stamp> stamp) {
		this.stamp = stamp;
	}
}
```

```
##--> 86B_OptionalExample_Person.java<-- ##

package com.udemy.app;

import java.util.Optional;

public class Person {
	private String name;
	private Optional<Passport> passport;

	public Person(String name, Optional<Passport> passport) {
		this.name = name;
		this.passport = passport;
	}

	public String getName() {
		return name;
	}

	public void setName(String name) {
		this.name = name;
	}

	public Optional<Passport> getPassport() {
		return passport;
	}

	public void setPassport(Optional<Passport> passport) {
		this.passport = passport;
	}


}
```

```
##--> 86B_OptionalExample_Stamp.java<-- ##

package com.udemy.app;

public class Stamp {
	private String countryOfStamp;

	public String getCountryOfStamp() {
		return countryOfStamp;
	}

	public void setCountryOfStamp(String countryOfStamp) {
		this.countryOfStamp = countryOfStamp;
	}

	public Stamp(String dateOfStamp) {
		this.countryOfStamp = dateOfStamp;
	}
}
```

```
##--> 88OperationOfStream_App.java<-- ##

package com.udemy.app;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class App {

      /**
       * Stream 1 - Operation of the Stream
       **/

      public static void main(String[] args) {

            /**
             * example 1
             */
            List<String> names = Arrays.asList("Alan", "James", "Kevin",
"Joe");
            List<String> filteredNames = names.stream().filter(name ->
name.startsWith("J")).collect(Collectors.toList());
            names.forEach(System.out::println);
            System.out.println(""); // new line
            filteredNames.forEach(System.out::println);

            /**
             * example 2
             */
//          System.out.println(""); //new line
//          names.stream().filter(name ->
name.startsWith("J")).forEach(System.out::println);
//          names.forEach(System.out::println);

            /**
             * example 3
             */
//          System.out.println("\n Stream example: ");
//          Stream<String> str = names.stream().filter(name ->
name.startsWith("J"));
//          str.forEach(System.out::println);

            /**
             * example 4
             */
//          str.forEach(System.out::println);

      }

}
```

```
##--> 89StreamMethods_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class App {

      /**
       * Stream 2 - Methods examples
       **/

      public static void main(String[] args) {

            /**
             * Stream : of / findFirst
             */
            System.out.println("- Stream : of / findFirst -");

            Stream.of("a1", "a2",
"a3").findFirst().ifPresent(System.out::println);

            /**
             * Stream : ofNullable
             */
            System.out.println("- Stream : ofNullable -");

            System.out.print("elements: ");
            Stream.ofNullable("Java").forEach(System.out::println);
            System.out.print("elements: ");
            //Stream.of(null).forEach(System.out::println);
//NullPointerException
            Stream.ofNullable(null).forEach(System.out::println);

            /**
             * Stream : count (& filter)
             */
            System.out.println("\n- Stream : count (& filter) -");

            List<String> names = Arrays.asList("Alan", "James", "Kevin",
"Joe");
            long count = names.stream().filter(name ->
name.startsWith("J")).count();
            System.out.println(count);

            /**
```

```
            * Stream - iterate
            */
           System.out.println("- Stream : iterate -");

           Stream<Integer> numbers = Stream.iterate(1, n -> n +
1).limit(10);
           numbers.forEach(n -> System.out.print(n + " "));

           /**
            * Stream - generate
            */
           System.out.println("\n- Stream : generate -");

           Stream<Double> randoms =
Stream.generate(Math::random).limit(3);
           randoms.forEach(System.out::println);

           /**
            * Stream - reduce
            */
           System.out.println("- Stream : reduce -");

           String[] title = { "Java", "Programing", "Step", "by",
"Step", "from", "A", "to", "Z", };
           // Using Arrays.stream() to convert array into Stream
           Stream<String> stream = Arrays.stream(title);
           Optional<String> courseTitle = stream.reduce((word1, word2) -
> word1 + "_" + word2);
           if (courseTitle.isPresent()) {
                 System.out.println(courseTitle.get());
           }

           /**
            * Stream - map
            */
           System.out.println("- Stream : map -");

           List<String> words = Arrays.asList("hello ", "java ",
"word");
           List<String> greeting =
words.stream().map(String::toUpperCase).collect(Collectors.toList());
           greeting.forEach(System.out::print);

           /**
            * Stream - flatMap
            */
           System.out.println("\n- Stream : flatMap -");

           List<String> femaleNames = Arrays.asList("Kate", "Judy",
"Nicole");
           List<String> maleNames = Arrays.asList("Kevin", "Peter",
"Joe");
           List<List<String>> listOfListofNames =
                Arrays.asList(femaleNames, maleNames);
```

```
            System.out.println("The Structure before flattening is : " +
                        listOfListofNames);

            List<String> listofNames  = listOfListofNames.stream()
                 .flatMap(list -> list.stream())
                 .collect(Collectors.toList());
            System.out.println("The Structure after flattening is : " +
                        listofNames);


      }

}
```

```
##--> 90FilteringMapWithStream_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class App {

      /**
       * Stream 3 - Filtering Map with Stream
       **/

      public static void main(String[] args) {

        /**
         * Stream - map, collect, filter, skip
         */

        Map<Integer, String> names = new HashMap<>();
        names.put(1, "Kevin");
        names.put(2, "Joe");
        names.put(3, "Judy");
        names.put(4, "Donald");
        names.put(5, "Kate");

        Map<Integer, String> collectedNames = names.entrySet().stream()
                .filter(map -> map.getKey() <= 4)
                .skip(2)
                .collect(Collectors.toMap(Map.Entry::getKey,
Map.Entry::getValue));

        System.out.println(collectedNames); //output : {3=Judy, 4=Donald}



      }

}
```

```
##--> 92SequentialVsParallelStream_App.java<-- ##

package com.udemy.app;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Iterator;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
import java.util.stream.Stream;

public class App {

      /**
       * Sequential vs Parallel Stream
       **/

      public static void main(String[] args) {

            double start = System.currentTimeMillis();
            long c = IntStream.rangeClosed(0, 1000000000)
                        .parallel()
                        .filter(i -> i % 2 == 0)
                        .count();
            double duration = (System.currentTimeMillis() - start);
            System.out.println("Counted: " + c + " in " + duration + "
milliseconds");
      }

}
```

```
##--> 94MultithreadingCreating1ThreadClass_App.java<-- ##

package com.udemy.app;

/**
 * Multithreading - Creating thread by extending Thread Class
 **/

class ThreadExampleClass extends Thread {
      public void run() {
            try {
                  System.out.println("Thread, number " +
Thread.currentThread().getId() + " is running");
            } catch (Exception e) {
                  System.out.println("Exception");
            }
      }
}

public class App {

      public static void main(String[] args) {

            int n = 6; // Number of threads
            for (int i = 0; i < n; i++) {
                  ThreadExampleClass object = new ThreadExampleClass();
                  object.start();
            }
      }
}
```

```
##--> 94MultithreadingCreating2RunnableInterface_App.java<-- ##

package com.udemy.app;

/**
 * Multithreading - Creating thread by implementing Runnable Interface
 **/

class ThreadExampleClass implements Runnable {
      public void run() {
            try {
                  System.out.println("Thread, number " +
Thread.currentThread().getId() + " is running");
            } catch (Exception e) {
                  System.out.println("Exception");
            }
      }
}

public class App {

      public static void main(String[] args) {

            int n = 6; // Number of threads
            for (int i = 0; i < n; i++) {
                  Thread object = new Thread(new ThreadExampleClass());
                  object.start();
            }
      }
}
```

```
##--> 95MultithreadingExample1_App.java<-- ##

package com.udemy.app;

/**
 * Multithreading example - Sequential process
 */

class Runner1 {

      public void startRunning() {
            for(int i=0;i<10;++i)
                  System.out.println("Runner1: "+i);
      }
}

class Runner2 {

      public void startRunning() {
            for(int i=0;i<10;++i)
                  System.out.println("Runner2: "+i);
      }
}

public class App {

      public static void main(String[] args) {

            Runner1 runner1 = new Runner1();
            Runner2 runner2 = new Runner2();

            runner1.startRunning();
            runner2.startRunning();
      }
}
```

```
##--> 95MultithreadingExample2_App.java<-- ##

package com.udemy.app;

/**
 * Multithreading example - Multithreading way
 */

class Runner1 extends Thread {

	@Override
	public void run() {
		for (int i = 0; i < 10; ++i) {
			System.out.println("Runner1: " + i);
//			try {
//				Thread.sleep(100);
//			} catch (InterruptedException e) {
//				e.printStackTrace();
//			}
		}
	}
}

class Runner2 extends Thread {

	@Override
	public void run() {
		for (int i = 0; i < 10; ++i) {
			System.out.println("Runner2: " + i);
//			try {
//				Thread.sleep(100);
//			} catch (InterruptedException e) {
//				e.printStackTrace();
//			}
		}
	}
}

public class App {

	public static void main(String[] args) throws InterruptedException
{

		Runner1 t1 = new Runner1();
		Runner2 t2 = new Runner2();

		t1.start();
		// t1.join();
		t2.start();
	}
}
```

```
##--> 97Synchronized_App.java<-- ##

package com.udemy.app;

/**
 * Synchronized
 */

public class App {

	private static int counter = 0;

	public static void increment() {
	//public synchronized static void increment() {
		++counter;
	}

	public static void process() {

		Thread t1 = new Thread(new Runnable() {

			@Override
			public void run() {
				for (int i = 0; i < 1000; ++i)
				increment();
			}
		});

		Thread t2 = new Thread(new Runnable() {

			@Override
			public void run() {
				for (int i = 0; i < 1000; ++i)
				increment();
			}
		});

		t1.start();
		t2.start();

		try {
			t1.join();
			t2.join();
		} catch (InterruptedException e) {
			e.printStackTrace();
		}
	}

	public static void main(String[] args) throws InterruptedException
{

		process();
		System.out.println(counter);
	}
```

}

```
##--> 98SynchronizedBlock_App.java<-- ##

package com.udemy.app;

/**
 * Synchronized Block
 */

class ShowNumbers {
	public void printTable(int n) {
		//synchronized (this) {
			System.out.println("Numbers start from " + n);
			for (int i = 1; i <= 5; i++) {
				System.out.println(n * i);
				try {
					Thread.sleep(400);
				} catch (Exception e) {
					System.out.println(e);
				}
			}
		//}
	}
}

class NumbersOneByOne extends Thread {
	ShowNumbers sn;
	NumbersOneByOne(ShowNumbers sn) {
		this.sn = sn;
	}
	public void run() {
		sn.printTable(1);
	}
}

class NumbersTensOf extends Thread {
	ShowNumbers sn;
	NumbersTensOf(ShowNumbers sn) {
		this.sn = sn;
	}
	public void run() {
		sn.printTable(10);
	}
}

public class App {
	public static void main(String args[]) {
		ShowNumbers obj = new ShowNumbers();
		NumbersOneByOne t1 = new NumbersOneByOne(obj);
		NumbersTensOf t2 = new NumbersTensOf(obj);
		t1.start();
		t2.start();
	}
}
```

```
##--> 99SynchronizedWaitNotify4_App.java<-- ##

package com.udemy.app;

/**
 * Synchronized - Wait, Notify
 */

class Processor {
	synchronized void waitingMethod() {
		Thread t = Thread.currentThread();
		System.out.println(t.getName() + " is relasing the lock and
going to wait");

		try {
			wait(); // releases the lock of this object and waits
		} catch (InterruptedException e) {
			e.printStackTrace();
		}

		System.out.println(t.getName() + " got the object lock back
and continues its execution");
	}

	synchronized void sleepingMethod() {
		Thread t = Thread.currentThread();

		try {
			Thread.sleep(3000);
		} catch (InterruptedException e) {
			e.printStackTrace();
		}

		notify(); // wakes up one thread randomly which is waiting
for lock of this object
		System.out.println("A thread which is waiting for lock of
this object is notified by " + t.getName());
	}
}


public class App {

	public static void main(String[] args) {
		final Processor p = new Processor();

		Thread t1 = new Thread() {
			public void run() {
				p.waitingMethod(); // t1 calling waitingMethod()
of 's' object
			}
		};
		Thread t2 = new Thread() {
			@Override
```

```
                public void run() {
                      p.sleepingMethod(); // t2 calling sleepingMethod()
of 's' object
                }
            };

            t1.start();
            t2.start();
      }
}
```

```
##--> 99SynchronizedWaitNotify_App.java<-- ##

package com.udemy.app;

/**
 * Synchronized - Wait, Notify
 */

class Processor {
	synchronized void waitingMethod() {
		Thread t = Thread.currentThread();
		System.out.println(t.getName() + " is relasing the lock and
going to wait");

		try {
			wait(); // releases the lock of this object and waits
		} catch (InterruptedException e) {
			e.printStackTrace();
		}

		System.out.println(t.getName() + " got the object lock back
and continues its execution");
	}

	synchronized void sleepingMethod() {
		Thread t = Thread.currentThread();

		try {
			Thread.sleep(3000);
		} catch (InterruptedException e) {
			e.printStackTrace();
		}

		notify(); // wakes up one thread randomly which is waiting
for lock of this object
		System.out.println("A thread which is waiting for lock of
this object is notified by " + t.getName());
	}
}


public class App {

	public static void main(String[] args) {
		final Processor p = new Processor();

		Thread t1 = new Thread() {
			public void run() {
				p.waitingMethod(); // t1 calling waitingMethod()
of 's' object
			}
		};
		Thread t2 = new Thread() {
			@Override
```

```
                public void run() {
                      p.sleepingMethod(); // t2 calling sleepingMethod()
of 's' object
                }
            };

            t1.start();
            t2.start();
      }
}
```